

Chapter 13

Clustering Algorithms

"Wherever you see a successful business, someone once made a courageous decision."

— Peter Drucker

Cluster analysis is a technique of partitioning a collection of unlabelled objects, with many attributes, into meaningful disjoint groups or clusters. This chapter aims to provide the basic concepts of clustering algorithms.

Learning Objectives

- Introduce the concepts of clustering
- Highlight the role of distance measures in clustering process
- Provide a taxonomy of clustering algorithms
- Explain hierarchical clustering algorithms
- Explain partitional clustering algorithms
- Briefly explain density-based, grid-based, and probabilistic model-based clustering techniques
- Discuss the validation techniques for clustering algorithms

13.1 INTRODUCTION TO CLUSTERING APPROACHES

Cluster analysis is the fundamental task of unsupervised learning. Unsupervised learning involves exploring the given dataset. Cluster analysis is a technique of partitioning a collection of unlabelled objects that have many attributes into meaningful disjoint groups or clusters. This is done using a trial and error approach as there are no supervisors available as in classification. The characteristic of clustering is that the objects in the clusters or groups are similar to each other within the clusters while differ from the objects in other clusters significantly.

The input for cluster analysis is examples or samples. These are known as objects, data points or data instances. All these terms are same and used interchangeably in this chapter. All the samples or objects with no labels associated with them are called unlabelled. The output is

the set of clusters (or groups) of similar data if it exists in the input. For example, the following Figure 13.1(a) shows data points or samples with two features shown in different shaded samples and Figure 13.1(b) shows the manually drawn ellipse to indicate the clusters formed.

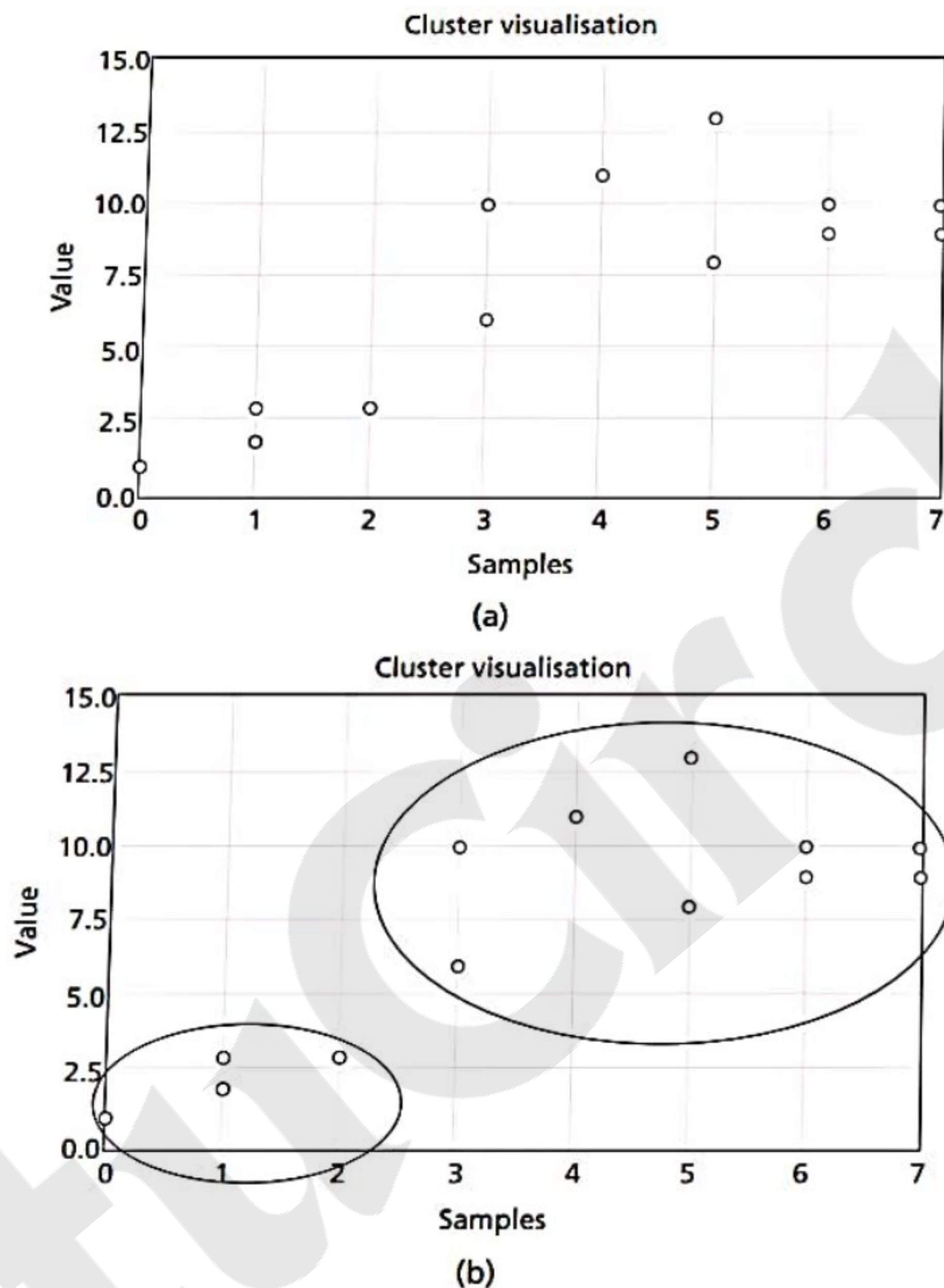


Figure 13.1: (a) Data Samples (b) Clusters' Description

Visual identification of clusters in this case is easy as the examples have only two features. But, when examples have more features, say 100, then clustering cannot be done manually and automatic clustering algorithms are required. Also, automating the clustering process is desirable as these tasks are considered difficult by humans and almost impossible. All clusters are represented by centroids. For example, if the input examples or data is (3, 3), (2, 6) and (7, 9), then the centroid is given as $\left(\frac{3+2+7}{3}, \frac{3+6+9}{3} \right) = (4, 6)$. The clusters should not overlap and every cluster should represent only one class. Therefore, clustering algorithms use trial and error method to form clusters that can be converted to labels. Thus, the important differences between classification and clustering are given in Table 13.1.

Table 13.1: Differences between Classification and Clustering

S.No.	Clustering	Classification
1.	Unsupervised learning and cluster formation are done by trial and error as there is no supervisor	Supervised learning with the presence of a supervisor to provide training and testing data
2.	Unlabelled data	Labelled data
3.	No prior knowledge in clustering	Knowledge of the domain is must to label the samples of the dataset
4.	Cluster results are dynamic	Once a label is assigned, it does not change

Applications of Clustering

1. Grouping based on customer buying patterns
2. Profiling of customers based on lifestyle
3. In information retrieval applications (like retrieval of a document from a collection of documents)
4. Identifying the groups of genes that influence a disease
5. Identification of organs that are similar in physiology functions
6. Taxonomy of animals, plants in Biology
7. Clustering based on purchasing behaviour and demography
8. Document indexing
9. Data compression by grouping similar objects and finding duplicate objects

Challenges of Clustering Algorithms

A huge collection of data with higher dimensions (i.e., features or attributes) can pose a problem for clustering algorithms. With the arrival of the internet, billions of data are available for clustering algorithms. This is a difficult task, as scaling is always an issue with clustering algorithms. Scaling is an issue where some algorithms work with lower dimension data but do not perform well for higher dimension data. Also, units of data can pose a problem, like some weights in kg and some in pounds can pose a problem in clustering. Designing a proximity measure is also a big challenge.

The advantages and disadvantages of the cluster analysis algorithms are given in Table 13.2.

Table 13.2: Advantages and Disadvantages of Clustering Algorithms

S.No.	Advantages	Disadvantages
1.	Cluster analysis algorithms can handle missing data and outliers.	Cluster analysis algorithms are sensitive to initialization and order of the input data.
2.	Can help classifiers in labelling the unlabelled data. Semi-supervised algorithms use cluster analysis algorithms to label the unlabelled data and then use classifiers to classify them.	Often, the number of clusters present in the data have to be specified by the user.

(Continued)

S.No.	Advantages	Disadvantages
3.	It is easy to explain the cluster analysis algorithms and to implement them.	Scaling is a problem.
4.	Clustering is the oldest technique in statistics and it is easy to explain. It is also relatively easy to implement.	Designing a proximity measure for the given data is an issue.

13.2 PROXIMITY MEASURES

Scan for 'Additional Information on Proximity Measures'



Clustering algorithms need a measure to find the similarity or dissimilarity among the objects to group them. Similarity and dissimilarity are collectively known as proximity measures. Often, the distance measures are used to find similarity between two objects, say i and j .

Distance measures are known as dissimilarity measures, as these indicate how one object is different from another. Measures like cosine similarity indicate the similarity among objects. Distance measures and similarity measures are two sides of the same coin, as more distance indicates more similarity and vice versa. Distance between two objects, say i and j , is denoted by the symbol D_{ij} .

The properties of the distance measures are:

1. D_{ij} is always positive or zero.
2. $D_{ij} = 0$, i.e., the distance between the object to itself is 0.
3. $D_{ij} = D_{ji}$. This property is called symmetry.
4. $D_{ij} \leq D_{ik} + D_{kj}$. This property is called triangular inequality.

If all these conditions are satisfied, then the distance measure is called a metric.

Based on the data types of the attributes of an object, distance measures vary. The concept of data types is discussed in Chapter 2. It can be recollected that the data types are divided into categorical and quantitative variables. Quantitative variables are numbers and are of two types – nominal and ordinal. For example, gender is a nominal variable as gender can be enumerated as $\text{Gender} = \{\text{male}, \text{female}\}$. Ordinal variables look like nominal variables but have an inherent order present in the enumeration. For example, temperature is a nominal variable as temperature can be enumerated as $\text{Temperature} = \{\text{low}, \text{medium}, \text{high}\}$. One can observe the inherent order present, that is, $\text{medium} > \text{low}$ and $\text{low} < \text{medium}$. Quantitative variables are real or Integer numbers or binary data. In binary data, the attributes of the object can take a Boolean value. Objects whose attributes take binary data are called binary objects.

Let us review some of the proximity measures.

Quantitative Variables

Some of the qualitative variables are discussed below.

Euclidean Distance It is one of the most important and common distance measures. It is also called as L_2 norm. It can be defined as the square root of squared differences between the coordinates of a pair of objects.

The Euclidean distance between objects x_i and x_j with k features is given as follows:

$$\text{Distance}(x_i, x_j) = \sqrt{\sum_{k=1}^n (x_{ik} - x_{jk})^2} \quad (13.1)$$

The advantage of Euclidean distance is that the distance does not change with the addition of new objects. But the disadvantage is that if the units change, the resulting Euclidean or squared Euclidean changes drastically. Another disadvantage is that as the Euclidean distance involves a square root and a square, the computational complexity is high for implementing the distance for millions or billions of operations involved.

City Block Distance City block distance is known as Manhattan distance. This is also known as boxcar, absolute value distance, Manhattan distance, Taxicab or L_1 norm. The formula for finding the distance is given as follows:

$$\text{Distance}(x_i, x_j) = \sum_{k=1}^n |x_{ik} - x_{jk}| \quad (13.2)$$

Chebyshev Distance Chebyshev distance is known as maximum value distance. This is the absolute magnitude of the differences between the coordinates of a pair of objects. This distance is called supremum distance or $L_{\max}$ or L_{∞} norm. The formula for computing Chebyshev distance is given as follows:

$$\text{Distance}(x_i, x_j) = \max_k |x_{ik} - x_{jk}| \quad (13.3)$$

Example 13.1: Suppose, if the coordinates of the objects are (0, 3) and (5, 8), then what is the Chebyshev distance?

Solution: The Euclidean distance using Eq. (13.1) is given as follows:

$$\begin{aligned} \text{Distance}(x_i, x_j) &= \sqrt{(0 - 5)^2 + (3 - 8)^2} \\ &= \sqrt{50} = 7.07 \end{aligned}$$

The Manhattan distance using Eq. (13.2) is given as follows:

$$\text{Distance}(x_i, x_j) = |(0 - 5) + (3 - 8)| = 10$$

The Chebyshev distance using Eq. (13.3) is given as follows:

$$\text{Max} \{|0 - 5|, |3 - 8|\} = \text{Max} \{5, 5\} = 5$$

Minkowski Distance In general, all the above distance measures can be generalized as:

$$\text{Distance}(x_i, x_j) = \left(\sum |x_{ik} - x_{jk}|^r \right)^{\frac{1}{r}} \quad (13.4)$$

This is called Minkowski distance. Here, r is a parameter. When the value of r is 1, the distance measure is called city block distance. When the value of r is 2, the distance measure is called Euclidean distance. When, r is ∞ , then this is Chebyshev distance.

Binary Attributes

Binary attributes have only two values. Distance measures discussed above cannot be applied to find distance between objects that have binary attributes. For finding the distance among objects with binary objects, the contingency Table 13.3 can be used. Let x and y be the objects consisting of N -binary objects. Then, the contingency table can be constructed by counting the number of matching of transitions, 0–0, 0–1, 1–0 and 1–1.

Table 13.3: Contingency Table

Attributes Matching	0	1
0	a	b
1	c	d

In other words, ' a ' is the number of attributes where x attribute is 0 and y attribute is 0. ' b ' is the number of attributes where x attribute is 0 and y attribute is 1, ' c ' is the number of attributes where x attribute is 1 and y attribute is 0 and ' d ' is the number of attributes where x attribute is 1 and y attribute is 1.

Simple Matching Coefficient (SMC) SMC is a simple distance measure and is defined as the ratio of number of matching attributes and the number of attributes. The formula is given as:

$$\frac{a + d}{a + b + c + d} \quad (13.5)$$

The values of a , b , c , and d can be observed from the Table 13.4.

Jaccard Coefficient Jaccard coefficient is another useful measure for and is given as follows:

$$J = \frac{d}{b + c + d} \quad (13.6)$$

Example 13.2: If the given vectors are $x = (1, 0, 0)$ and $y = (1, 1, 1)$ then find the SMC and Jaccard coefficient?

Solution: It can be seen from Table 13.2 that, $a = 0$, $b = 2$, $c = 0$ and $d = 1$.

The SMC using Eq. (13.5) is given as $\frac{a + d}{a + b + c + d} = \frac{0 + 1}{0 + 2 + 0 + 1} = 0.33$

Jaccard coefficient using Eq. (13.6) is given as $J = \frac{d}{b + c + d} = \frac{1}{2 + 0 + 1} = 0.33$

Hamming Distance Hamming distance is another useful measure that can be used for knowing the sequence of characters or binary values. It indicates the number of positions at which the characters or binary bits are different.

For example, the hamming distance between $x = (1\ 0\ 1)$ and $y = (1\ 1\ 0)$ is 2 as x and y differ in two positions. The distance between two words, say wood and hood is 1, as they differ in only one character. Sometimes, more complex distance measures like edit distance can also be used.

Categorical Variables

In many cases, categorical values are used. It is just a code or symbol to represent the values. For example, for the attribute Gender, a code 1 can be given to female and 0 can be given to male.

To calculate the distance between two objects represented by variables, we need to find only whether they are equal or not. This is given as:

$$\text{Distance}(x, y) = \begin{cases} 1 & \text{if } x \neq y \\ 0 & \text{if } x = y \end{cases} \quad (13.7)$$

Ordinal Variables

Ordinal variables are like categorical values but with an inherent order. For example, designation is an ordinal variable. If job designation is 1 or 2 or 3, it means code 1 is higher than 2 and code 2 is higher than 3. It is ranked as $1 > 2 > 3$.

Let us assume the designations of office employees are clerk, supervisor, manager and general manager. These can be designated as numbers as clerk = 1, supervisor = 2, manager = 3 and general manager = 4. Then, the distance between employee X who is a clerk and Y who is a manager can be obtained as:

$$\text{Distance}(X, Y) = \frac{|\text{position}(X) - \text{position}(Y)|}{n - 1} \quad (13.8)$$

Here, position(X) and position(Y) indicate the designated numerical value. Thus, the distance between X (Clerk = 1) and Y (Manager = 3) using Eq. (13.8) is given as:

$$\text{Distance}(X, Y) = \frac{|\text{position}(X) - \text{position}(Y)|}{n - 1} = \frac{|1 - 3|}{4 - 1} = \frac{2}{3} \approx 0.66$$

Vector Type Distance Measures

For text classification, vectors are normally used. Cosine similarity is a metric used to measure how similar the documents are irrespective of their size. Cosine similarity measures the cosine of the angle between two vectors projected in a multi-dimensional space. The similarity function for vector objects can be defined as:

$$\text{sim}(X, Y) = \frac{X \cdot Y}{\|X\| \times \|Y\|} = \frac{\sum_{i=1}^n X_i \times Y_i}{\sqrt{\sum_{i=1}^n X_i^2} \times \sqrt{\sum_{i=1}^n Y_i^2}} \quad (13.9)$$

The numeration is the dot product of the vectors A and B. The denominator is the product of the norm of vectors A and B.

Example 13.3: If the given vectors are $A = \{1, 1, 0\}$ and $B = \{0, 1, 1\}$, then what is the cosine similarity?

Solution: The dot product of the vector is $1 \times 0 + 1 \times 1 + 0 \times 1 = 1$. The norm of the vectors A and B is $\sqrt{2}$.

So, the cosine similarity using Eq. (13.9) is given as $\frac{1}{\sqrt{2}\sqrt{2}} = \frac{1}{2} = 0.5$

Now, let us discuss the types of clustering algorithms, which include hierarchical, partitional, density-based and grid-based algorithms.

Scan for information on 'Taxonomy of Clustering Algorithms'



13.3 HIERARCHICAL CLUSTERING ALGORITHMS

Hierarchical methods produce a nested partition of objects with hierarchical relationships among objects. Often, the hierarchy relationship is shown in the form of a dendrogram.

Hierarchical methods include categories, agglomerative methods and divisive methods. In agglomerative methods, initially all individual samples are considered as a cluster, that is, a cluster with a single element. Then, they are merged and the process is continued to get a single cluster. Divisive methods use another kind of philosophy, where a single cluster of all samples of the dataset taken initially is chosen and then partitioned. This partition process is continued until the cluster is split into smaller clusters.

Agglomerative methods merge clusters to reduce the number of clusters. This is repeated each time while merging two closest clusters to get a single cluster. The procedure of agglomerative clustering is given as follows:

Algorithm 13.1: Agglomerative Clustering

1. Place each N sample or data instance into a separate cluster. So, initially N clusters are available.
2. Repeat the following steps until a single cluster is formed:
 - (a) Determine two most similar clusters.
 - (b) Merge the two clusters into a single cluster reducing the number of clusters as $N-1$.
3. Choose resultant clusters of step 2 as result.

All the clusters that are produced by hierarchical algorithms have equal diameters. The main disadvantage of this approach is that once the cluster is formed, it is an irreversible decision.

13.3.1 Single Linkage or MIN Algorithm

Hierarchical clustering algorithms produce nested clusters, which can be visualized as a hierarchical tree or dendrogram. The idea behind this approach is proximity among clusters. In single linkage algorithm, the smallest distance (x, y) , where x is from one cluster and y is from another cluster, is the distance between all possible pairs of the two groups or clusters (or simply the smallest distance of two points where points are in different clusters) and is used for merging the clusters. This corresponds to finding of minimum spanning tree (MST) of a graph.

The distance measures between individual samples or data points is already demonstrated in the previous section 13.2. To understand the single linkage algorithm, go through the following numerical problem that involves finding the distance between clusters.

Example 13.4: Consider the array of points as shown in the following Table 13.4.

Table 13.4: Sample Data

Objects	X	Y
0	1	4
1	2	8
2	5	10
3	12	18
4	14	28

Apply single linkage algorithm.

Solution: The similarity table among the variables is computed as shown in Table 13.4. Euclidean distance is computed as shown in the following Table 13.5.

Table 13.5: Proximity Matrix

Objects	0	1	2	3	4
0	–	4.123	7.211	17.804	27.295
1		–	3.606	14.142	23.324
2			–	10.630	20.124
3				–	10.198
4					–

The minimum distance is 3.606. Therefore, the items 1 and 2 are clustered together. The resultant is shown in Table 13.6.

Table 13.6: After Iteration 1

Clusters	{1, 2}	0	3	4
{1, 2}	–	4.123	10.630	20.124
0		–	17.804	27.295
3			–	10.198
4				–

The distance between the group {1, 2} and items {0}, {3} and {4} is computed using the formula:

$$D_{SL}(C_i, C_j) = \text{minimum}_{a \in C_i, b \in C_j} d(a, b) \quad (13.10)$$

Here, D_{SL} is the single linkage distance, C_i, C_j are clusters and $d(a, b)$ is the distance between the elements a and b .

Thus, the distance between {1, 2} and {0} is:

$$\text{Minimum} \{d(1, 0), d(2, 0)\} = \text{Minimum} \{4.123, 7.211\} = 4.123$$

The distance between {1, 2} and {3} is given as:

$$\text{Minimum} \{d(1, 3), d(2, 3)\} = \text{Minimum} \{14.142, 10.630\} = 10.630$$

The distance between {1, 2} and {4} is given as:

$$\text{Minimum} \{d(1, 4), d(2, 4)\} = \text{Minimum} \{23.324, 20.124\} = 20.124$$

This is shown in Table 13.6. The minimum distance of Table 13.6 is 4.123.

Therefore, {0, 1, 2} is clustered together. This result is shown in Table 13.7.

Table 13.7: After Iteration 2

Clusters	{0, 1, 2}	3	4
{0, 1, 2}	–	10.630	20.124
3		–	10.198
4			–

Thus, the distance between {0, 1, 2} and {3} using Eq. (13.10) is given as

$$\text{Minimum } \{ \{0, 3\}, \{1, 3\}, \{2, 3\} \} = \text{Minimum } \{ 17.804, 14.142, 10.630 \} = 10.630.$$

Thus, the distance between {0, 1, 2} and {4} is:

$$\text{Minimum } \{ \{0, 4\}, \{1, 4\}, \{2, 4\} \} = \text{Minimum } \{ 27.295, 23.324, 20.124 \} = 20.124.$$

This is shown in Table 13.7. The minimum is 10.198.

Therefore, the items {3, 4} are merged. And, the items {1, 2, 3} and {4, 5} are merged. The resultant is shown in Table 13.8.

Table 13.8: After Iteration 3

Clusters	{0, 1, 2}	{3, 4}
{0, 1, 2}	–	?
{3, 4}		–

The computation of ? in Table 13.8 is not needed as no other items are left. Therefore, the clusters {0, 1, 2} and {3, 4} are merged.

Dendrograms are used to plot the hierarchy of clusters. Dendrogram for the above clustering is shown in the following Figure 13.2.

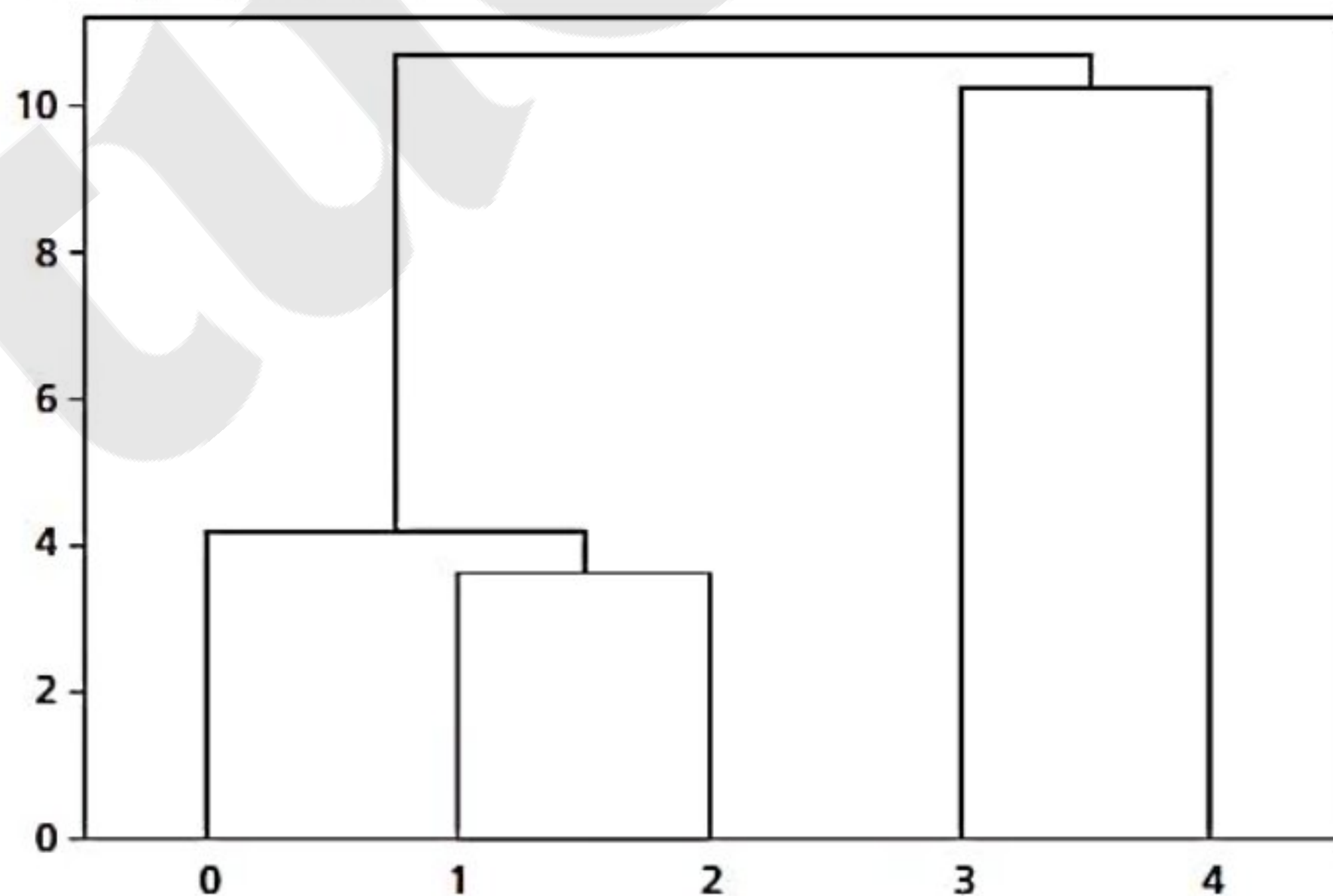


Figure 13.2: Dendrogram for Table 13.4

13.3.2 Complete Linkage or MAX or Clique

In complete linkage algorithm, the distance (x, y) , (where x is from one cluster and y is from another cluster), is the largest distance between all possible pairs of the two groups or clusters (or simply the largest distance of two points where points are in different clusters) as given below. It is used for merging the clusters.

$$D_{CL}(C_i, C_j) = \text{maximum}_{a \in C_i, b \in C_j} d(a, b) \quad (13.11)$$

Dendrogram for the above clustering is shown in Figure 13.3.

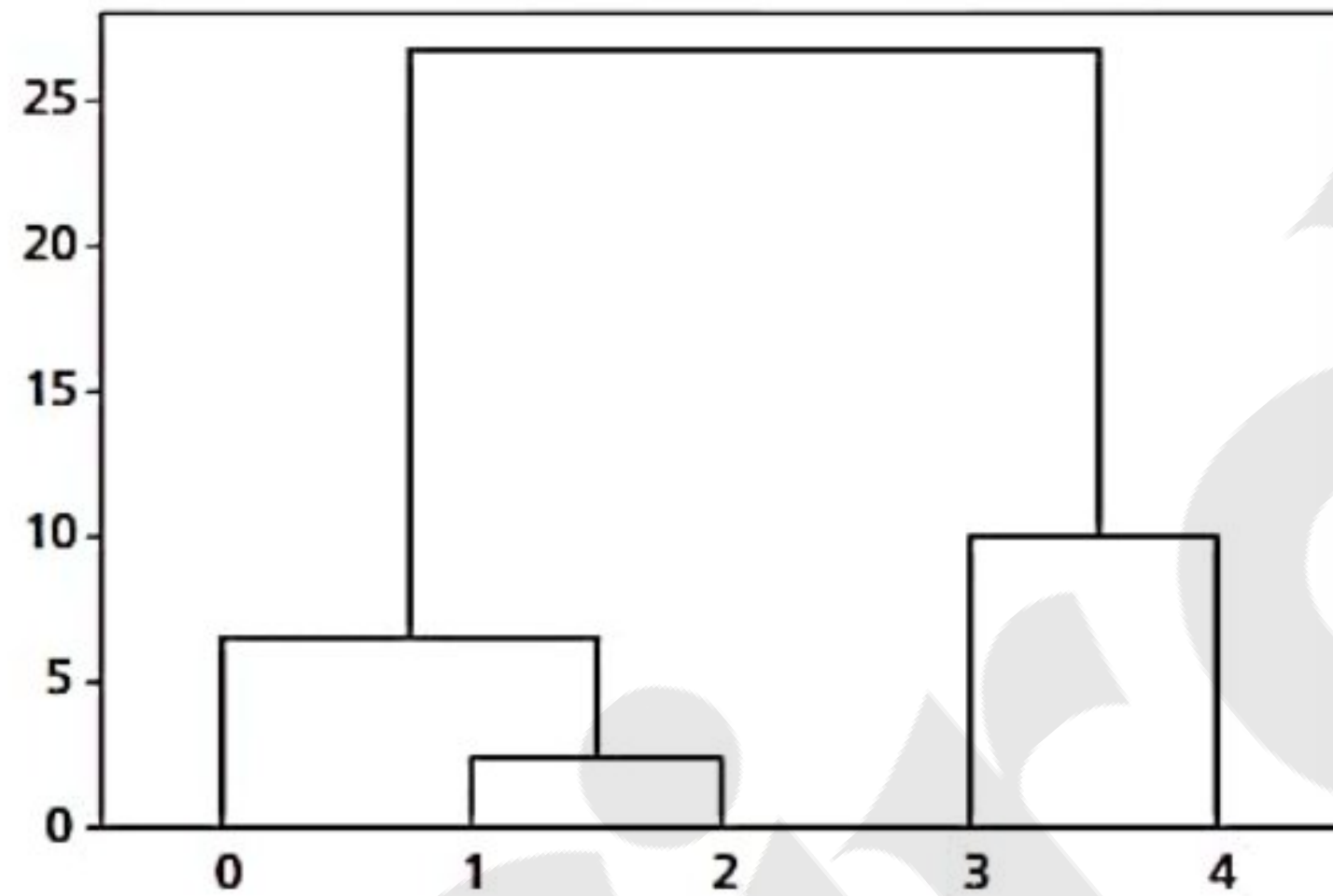


Figure 13.3: Dendrogram for Table 13.4

13.3.3 Average Linkage

In case of an average linkage algorithm, the average distance of all pairs of points across the clusters is used to form clusters. The average value computed between clusters c_i, c_j is given as follows:

$$D_{AL}(C_i, C_j) = \frac{1}{m_i m_j} \sum_{a \in C_i, b \in C_j} d(a, b) \quad (13.12)$$

Here, m_i and m_j are sizes of the clusters.

The dendrogram for Table 13.4 is given in Figure 13.4.

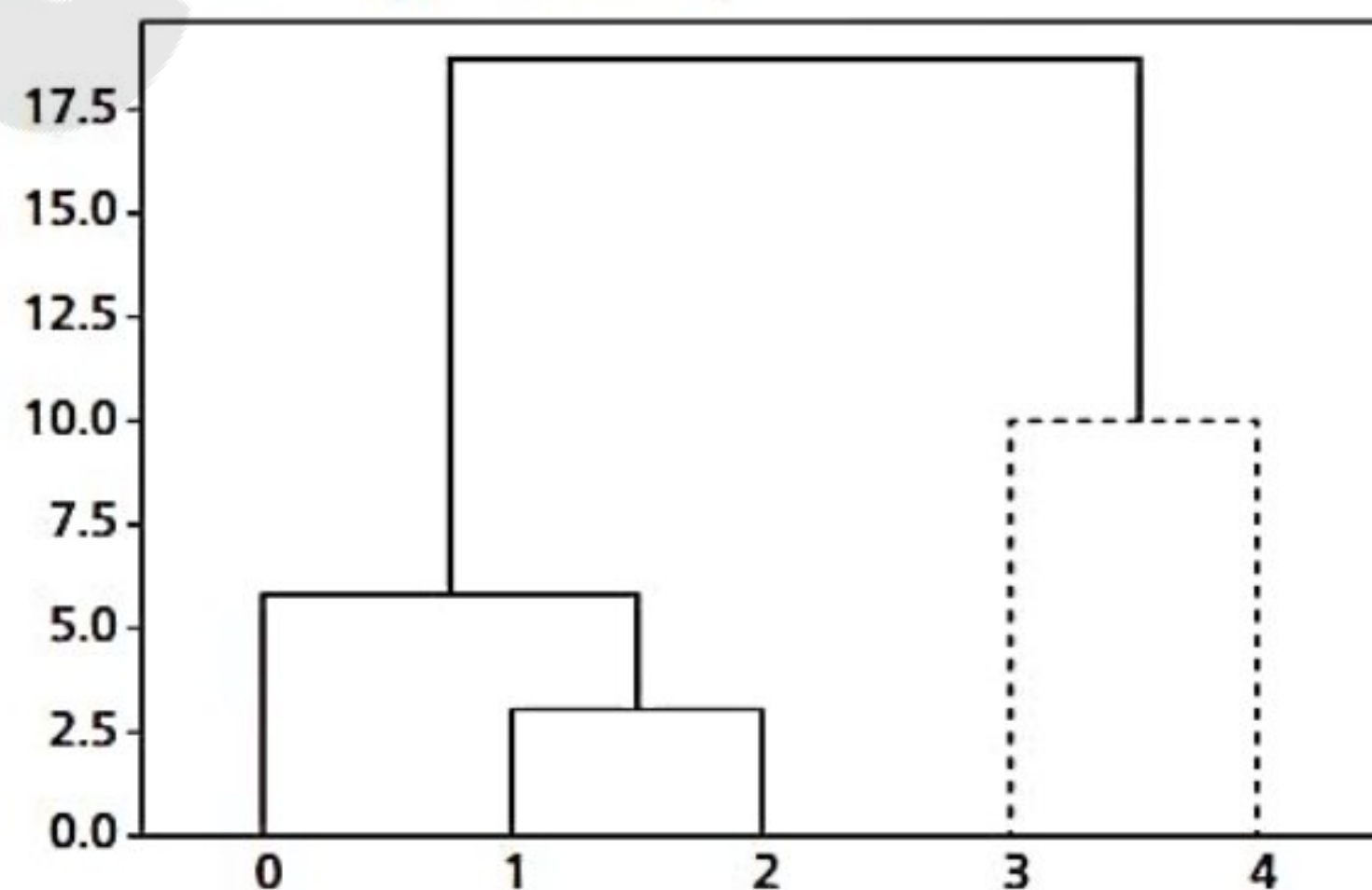


Figure 13.4: Dendrogram for Table 13.4

13.3.4 Mean-Shift Clustering Algorithm

Mean-shift is a non-parametric and hierarchical clustering algorithm. This algorithm is also known as mode seeking algorithm or a sliding window algorithm. It has many applications in image processing and computer vision.

There is no need for any prior knowledge of clusters or shape of the clusters present in the dataset. The algorithm slowly moves from its initial position towards the dense regions.

The algorithm uses a window, which is basically a weighting function. Gaussian window is a good example of a window. The radius of the kernel is called bandwidth. The entire window is called a kernel. The window is based on the concept of kernel density function and its aim is to find the underlying data distribution. The method of calculation of mean is dependent on the choice of windows. If a Gaussian window is chosen, then every point is assigned a weight that decreases as the distance from the kernel center increases. The algorithm is given below.

Algorithm 13.2: Mean-Shift Clustering

Step 1: Design a window.

Step 2: Place the window on a set of data points.

Step 3: Compute the mean for all the points that come under the window.

Step 4: Move the center of the window to the mean computed in step 3. Thus, the window moves towards the dense regions. The movement to the dense region is controlled by a mean shift vector. The mean shift vector is given as:

$$v_s = \frac{1}{K} \sum_{x_i \in S_k} (x_i - x) \quad (13.13)$$

Here, K is the number of points and S_k is the data points where the distance from data points x_i and centroid of the kernel x is within the radius of the sphere. Then, the centroid is updated as $x = x + v_s$.

Step 5: Repeat the steps 3–4 for convergence. Once convergence is achieved, no further points can be accommodated.

Advantages

1. No model assumptions
2. Suitable for all non-convex shapes
3. Only one parameter of the window, that is, bandwidth is required
4. Robust to noise
5. No issues of local minima or premature termination

Disadvantages

1. Selecting the bandwidth is a challenging task. If it is larger, then many clusters are missed. If it is small, then many points are missed and convergence occurs as the problem.
2. The number of clusters cannot be specified and user has no control over this parameter.

Scan for 'Additional Examples'



13.4 PARTITIONAL CLUSTERING ALGORITHM

'k-means' algorithm is a straightforward iterative partitional algorithm. Here, k stands for the user specified requested clusters as users are not aware of the clusters that are present in the dataset. The k -means algorithm assumes that the clusters do not overlap. Therefore, a sample or data point can belong to only one cluster in the end. Also, this algorithm can detect clusters of shapes like circular or spherical.

Initially, the algorithm needs to be initialized. The algorithm can select k data points randomly or use the prior knowledge of the data. In most cases, in k -means algorithm setup, prior knowledge is absent. The composition of the cluster is based on the initial condition, therefore, initialization is an important task. The sample or data points need to be normalized for better performance. The concepts of normalization are covered in Chapter 3.

The core process of the k -mean algorithm is assigning a sample to a cluster, that is, assigning each sample or data point to the k cluster centers based on its distance and the centroid of the clusters. This distance should be minimum. As a new sample is added, new computation of mean vectors of the points for that cluster to which sample is assigned is required. Therefore, this iterative process is continued until no change of instances to clusters is noticed. This algorithm then terminates and the termination is guaranteed.

Algorithm 13.3: k-means Algorithm

Step 1: Determine the number of clusters before the algorithm is started. This is called k .

Step 2: Choose k instances randomly. These are initial cluster centers.

Step 3: Compute the mean of the initial clusters and assign the remaining sample to the closest cluster based on Euclidean distance or any other distance measure between the instances and the centroid of the clusters.

Step 4: Compute new centroid again considering the newly added samples.

Step 5: Perform the steps 3–4 till the algorithm becomes stable with no more changes in assignment of instances and clusters.

k -means can also be viewed as greedy algorithm as it involves partitioning n samples to k clusters to minimize Sum of Squared Error (SSE). SSE is a metric that is a measure of error that gives the sum of the squared Euclidean distances of each data to its closest centroid. It is given as:

$$SSE = \sum_{i=1}^k \text{dist}(c_i, x)^2 \quad (13.14)$$

Here, c_i is the centroid of the i^{th} cluster, x is the sample or data point and dist is the Euclidean distance. The aim of the k -means algorithm is to minimize SSE.

Advantages

1. Simple
2. Easy to implement

Disadvantages

1. It is sensitive to initialization process as change of initial points leads to different clusters.
2. If the samples are large, then the algorithm takes a lot of time.

How to Choose the Value of k ?

It is obvious that k is the user specified value specifying the number of clusters that are present. Obviously, there are no standard rules available to pick the value of k . Normally, the k -means algorithm is run with multiple values of k and within group variance (sum of squares of samples with its centroid) and plotted as a line graph. This plot is called Elbow curve. The optimal or best value of k can be determined from the graph. The optimal value of k is identified by the flat or horizontal part of the Elbow curve.

Complexity

The complexity of k -means algorithm is dependent on the parameters like n , the number of samples, k , the number of clusters, $\Theta(nkId)$. I is the number of iterations and d is the number of attributes. The complexity of k -means algorithm is $O(n^2)$.

Example 13.5: Consider the following set of data given in Table 13.9. Cluster it using k -means algorithm with the initial value of objects 2 and 5 with the coordinate values (4, 6) and (12, 4) as initial seeds.

Table 13.9: Sample Data

Objects	X-coordinate	Y-coordinate
1	2	4
2	4	6
3	6	8
4	10	4
5	12	4

Solution: As per the problem, choose the objects 2 and 5 with the coordinate values. Hereafter, the objects' id is not important. The samples or data points (4, 6) and (12, 4) are started as two clusters as shown in Table 13.10.

Initially, centroid and data points are same as only one sample is involved.

Table 13.10: Initial Cluster Table

Cluster 1	Cluster 2
(4, 6)	(12, 4)
Centroid 1 (4, 6)	Centroid 2 (12, 4)

Iteration 1: Compare all the data points or samples with the centroid and assign to the nearest sample. Take the sample object 1 (2, 4) from Table 13.9 and compare with the centroid of

the clusters in Table 13.10. The distance is 0. Therefore, it remains in the same cluster. Similarly, consider the remaining samples. For the object 1 (2, 4), the Euclidean distance between it and the centroid is given as:

$$\text{Dist (1, centroid 1)} = \sqrt{(2-4)^2 + (4-6)^2} = \sqrt{8}$$

$$\text{Dist (1, centroid 2)} = \sqrt{(2-12)^2 + (4-4)^2} = \sqrt{100} = 10$$

Object 1 is closer to the centroid of cluster 1 and hence assign it to cluster 1. This is shown in Table 13.11. Object 2 is taken as centroid point.

For the object 3 (6, 8), the Euclidean distance between it and the centroid points is given as:

$$\text{Dist (3, centroid 1)} = \sqrt{(6-4)^2 + (8-6)^2} = \sqrt{8}$$

$$\text{Dist (3, centroid 2)} = \sqrt{(6-12)^2 + (8-4)^2} = \sqrt{52}$$

Object 3 is closer to the centroid of cluster 1 and hence remains in the same cluster 1.

Proceed with the next point object 4(10, 4) and again compare it with the centroids in Table 13.10.

$$\text{Dist (4, centroid 1)} = \sqrt{(10-4)^2 + (4-6)^2} = \sqrt{40}$$

$$\text{Dist (4, centroid 2)} = \sqrt{(10-12)^2 + (4-4)^2} = \sqrt{4} = 2$$

Object 4 is closer to the centroid of cluster 2 and hence assign it to the cluster table. Object 4 is in the same cluster. The final cluster table is shown in Table 13.11.

Obviously, Object 5 is in Cluster 3. Recompute the new centroids of cluster 1 and cluster 2. They are (4, 6) and (11, 4), respectively.

Table 13.11: Cluster Table After Iteration 1

Cluster 1	Cluster 2
(4, 6)	(10, 4)
(2, 4)	(12, 4)
(6, 8)	
Centroid 1 (4, 6)	Centroid 2 (11, 4)

The second iteration is started again with the Table 13.11.

Obviously, the point (4, 6) remains in cluster 1, as the distance of it with itself is 0. The remaining objects can be checked. Take the sample object 1 (2, 4) and compare with the centroid of the clusters in Table 13.12.

$$\text{Dist (1, centroid 1)} = \sqrt{(2-4)^2 + (4-6)^2} = \sqrt{8}$$

$$\text{Dist (1, centroid 2)} = \sqrt{(2-11)^2 + (4-4)^2} = \sqrt{81} = 9$$

Object 1 is closer to centroid of cluster 1 and hence remains in the same cluster. Take the sample object 3 (6, 8) and compare with the centroid values of clusters 1 (4, 6) and cluster 2 (11, 4) of the Table 13.12.

$$\text{Dist (3, centroid 1)} = \sqrt{(6-4)^2 + (8-6)^2} = \sqrt{8}$$

$$\text{Dist (3, centroid 2)} = \sqrt{(6-11)^2 + (8-4)^2} = \sqrt{41}$$

Object 3 is closer to centroid of cluster 1 and hence remains in the same cluster. Take the sample object 4 (10, 4) and compare with the centroid values of clusters 1 (4, 6) and cluster 2 (11, 4) of the Table 13.12:

$$\text{Dist (4, centroid 1)} = \sqrt{(10 - 4)^2 + (4 - 6)^2} = \sqrt{40}$$

$$\text{Dist (3, centroid 2)} = \sqrt{(10 - 11)^2 + (4 - 4)^2} = \sqrt{1} = 1$$

Object 3 is closer to centroid of cluster 2 and hence remains in the same cluster. Obviously, the sample (12, 4) is closer to its centroid as shown below:

$$\text{Dist (5, centroid 1)} = \sqrt{(12 - 4)^2 + (4 - 6)^2} = \sqrt{68}$$

$\text{Dist (5, centroid 2)} = \sqrt{(12 - 11)^2 + (4 - 4)^2} = \sqrt{1} = 1$. Therefore, it remains in the same cluster. Object 5 is taken as centroid point.

The final cluster Table 13.12 is given below:

Table 13.12: Cluster Table After Iteration 2

Cluster 1	Cluster 2
(4, 6)	(10, 4)
(2, 4)	(12, 4)
(6, 8)	
Centroid (4, 6)	Centroid (11, 4)

There is no change in the cluster Table 13.12. It is exactly the same; therefore, the *k*-means algorithm terminates with two clusters with data points as shown in the Table 13.12.

Scan for 'Additional Examples'



13.5 DENSITY-BASED METHODS

Density-based spatial clustering of applications with noise (DBSCAN) is one of the density-based algorithms. Density of a region represents the region where many points above the specified threshold are present. In a density-based approach, the clusters are regarded as dense regions of objects that are separated by regions of low density such as noise. This is same as a human's intuitive way of observing clusters.

The concept of density and connectivity is based on the local distance of neighbours. The functioning of this algorithm is based on two parameters, the size of the neighbourhood (ϵ) and the minimum number of points (m).

1. Core point – A point is called a core point if it has more than specified number of points (m) within ϵ -neighbourhood.
2. Border point – A point is called a border point if it has fewer than ' m ' points but is a neighbour of a core point.
3. Noise point – A point that is neither a core point nor border point.

The main idea is that every data point or sample should have at least a minimum number of neighbours in a neighbourhood. The neighbourhood of radius ϵ should have at least m points. The notion of density connectedness determines the quality of the algorithm.

The following connectedness measures are used for this algorithm.

1. Direct density reachable – The point X is directly reachable from Y , if:
 - (a) X is the ϵ -neighborhood of Y
 - (b) Y is a core point
2. Densely reachable – The point X is densely reachable from Y , if there is a set of core points that leads from Y to X .
3. Density connected – X and Y are densely connected if Z is a core point and thus points X and Y are densely reachable from Z .

Algorithm 13.4: DBSCAN

Step 1: Randomly select a point p . Compute distance between p and all other points.

Step 2: Find all points from p with respect to its neighbourhood and check whether it has minimum number of points m . If so, it is marked as a core point.

Step 3: If it is a core point, then a new cluster is formed, or existing cluster is enlarged.

Step 4: If it is a border point, then the algorithm moves to the next point and marks it as visited.

Step 5: If it is a noise point, they are removed.

Step 6: Merge the clusters if it is mergeable, $\text{dist}(c_i, c_j) < \epsilon$.

Step 7: Repeat the process 3–6 till all points are processed.

Advantages

1. No need for specifying the number of clusters beforehand
2. The algorithm can detect clusters of any shapes
3. Robust to noise
4. Few parameters are needed

The complexity of this algorithm is $O(n \log n)$.

13.6 GRID-BASED APPROACH

Grid-based approach is a space-based approach. It partitions space into cells, the given data is fitted on the cells for cluster formation.

There are three important concepts that need to be mastered for understanding the grid-based schemes. They are:

1. Subspace clustering
2. Concept of dense cells
3. Monotonicity property

Let us discuss about them.

Subspace Clustering

Grid-based algorithms are useful for clustering high-dimensional data, that is, data with many attributes. Some data like gene data may have millions of attributes. Every attribute is called a dimension. But all the attributes are not needed, as in many applications one may not require all the attributes. For example, an employee's address may not be required for profiling his diseases. Age may be required in that case. So, one can conclude that only a subset of features is required. For example, one may be interested in grouping gene data with similar characteristics or organs that have similar functions.

Finding subspaces is difficult. For example, N dimensions may have 2^{N-1} subspaces. Exploring all the subspaces is a difficult task. Here, only the CLIQUE algorithms are useful for exploring the subspaces. CLIQUE (Clustering in Quest) is a grid-based method for finding clustering in subspaces. CLIQUE uses a multiresolution grid data structure.

Concept of Dense Cells

CLIQUE partitions each dimension into several overlapping intervals and intervals it into cells. Then, the algorithm determines whether the cell is dense or sparse. The cell is considered dense if it exceeds a threshold value, say τ . Density is defined as the ratio of number of points and volume of the region. In one pass, the algorithm finds the number of cells, number of points, etc. and then combines the dense cells. For that, the algorithm uses the contiguous intervals and a set of dense cells.

Algorithm 13.5: Dense Cells

- Step 1:** Define a set of grid points and assign the given data points on the grid.
- Step 2:** Determine the dense and sparse cells. If the number of points in a cell exceeds the threshold value τ , the cell is categorized as dense cell. Sparse cells are removed from the list.
- Step 3:** Merge the dense cells if they are adjacent.
- Step 4:** Form a list of grid cells for every subspace as output.

Monotonicity Property

CLIQUE uses anti-monotonicity property or apriori property of the famous apriori algorithm. It means that all the subsets of a frequent item should be frequent. Similarly, if the subset is infrequent, then all its supersets are infrequent as well. Based on the apriori property, one can conclude that a k -dimensional cell has r points if and only if every $(k - 1)$ dimensional projections of this cell have at least r points. So like association rule mining that uses apriori rule, the candidate dense cells are generated for higher dimensions. The algorithm works in two stages as shown below.

Algorithm 13.6: CLIQUE

- Stage 1:**
 - Step 1:** Identify the dense cells.
 - Step 2:** Merge dense cells c_1 and c_2 if they share the same interval.

(Continued)

Step 3: Generate Apriori rule to generate $(k + 1)^{\text{th}}$ cell for higher dimension. Then, check whether the number of points cross the threshold. This is repeated till there are no dense cells or new generation of dense cells.

Stage 2:

Step 1: Merging of dense cells into a cluster is carried out in each subspace using maximal regions to cover dense cells. The maximal region is an hyperrectangle where all cells fall into.

Step 2: Maximal region tries to cover all dense cells to form clusters.

In stage two, CLIQUE starts from dimension 2 and starts merging. This process is continued till the n -dimension.

Advantages of CLIQUE

1. Insensitive to input order of objects
2. No assumptions of underlying data distributions
3. Finds subspace of higher dimensions such that high-density clusters exist in those subspaces

Disadvantage

The disadvantage of CLIQUE is that tuning of grid parameters, such as grid size, and finding optimal threshold for finding whether the cell is dense or not is a challenge.



Chapter 14

Reinforcement Learning

"Properly used positive reinforcement is extremely powerful."
— B.F. Skinner

Reinforcement learning is a branch of machine learning. There are many applications of reinforcement learning such as games, robotics, and resource management. This chapter aims to provide the basics of reinforcement learning.

Learning Objectives

- Get an overview of Reinforcement Learning
- Provide a mathematical formulation of Reinforcement problem
- Learn the basics of Markov decision process
- Introduce the basics of Dynamic Programming
- Know about Model free methods
- Introduce Q-Learning Algorithm
- Gain knowledge about SARSA algorithm

14.1 OVERVIEW OF REINFORCEMENT LEARNING

Reinforcement learning (RL) mimics human beings and is a branch of machine learning. Humans observe the environment through senses such as eye and ear. The inputs are processed by brain. The brain then suggests the actions and acts voluntarily or involuntarily.

In humans, learning happens in a real world called environment. Take a kid for an example. How does the kid learn? The kid interacts with the environment and gains valuable experience. Experience leads to learning. For example, when a kid accidentally burns his hand in fire, he learns to stay away from it in the future and be careful. Similarly, the kid learns to do good things repeatedly by the encouragement or gifts given by parents or teachers. Awards can be positive or negative. Awarding a kid is a positive reinforcement and scolding is a negative reward or deterrent as shown in Figure 14.1. Thus, kids gain experience and learn many things through rewards and punishments (negative rewards).



Figure 14.1: Examples of Positive and Negative Rewards

Scan for 'Additional Information'



In computers, these scenarios can be simulated in many ways. As a kid executes many actions and receives a mix of positive and negative rewards that lead to gaining experience and learning, a computer program or robot can learn through experiences of simulated scenarios. Thus, reinforcement learning is a mathematical framework for learning. There are two types of reinforcement learning – positive and negative. Positive reinforcement learning is a recurrence of behaviour due to positive rewards. Rewards increase strength and the frequency of a specific behaviour. This encourages to execute similar actions that yield maximum reward. Similarly, in negative reinforcement learning, negative rewards are used as a deterrent to weaken the behaviour and to avoid it. In a maze game, there may a danger spot that may lead to loss. Negative rewards can be designed for such spots so that the agent does not visit that spot. Positive and negative rewards are simulated in reinforcement learning, say +10 for positive reward and -10 for some danger or negative reward.

Reinforcement learning is an example of semi-supervised learning technique and is used to model sequential decision-making process.

14.2 SCOPE OF REINFORCEMENT LEARNING

What are the situations where reinforcement learning can be used? Consider the following grid game shown in Figure 14.2, where a robot can move. Assume the starting node is *E* and goal node is *G*, the game is about finding the shortest path from starting to goal state.

Consider another grid game as shown in Figure 14.3.

In this grid game, the grey tile indicates the danger, black is a block and the tile with diagonal lines is the goal. The aim is to start, say from bottom-left grid, using the actions left, right, top and bottom to reach the goal state.

Reinforcement learning is highly suitable for solving problems like this, especially the ones with uncertainty. Reinforcement is not

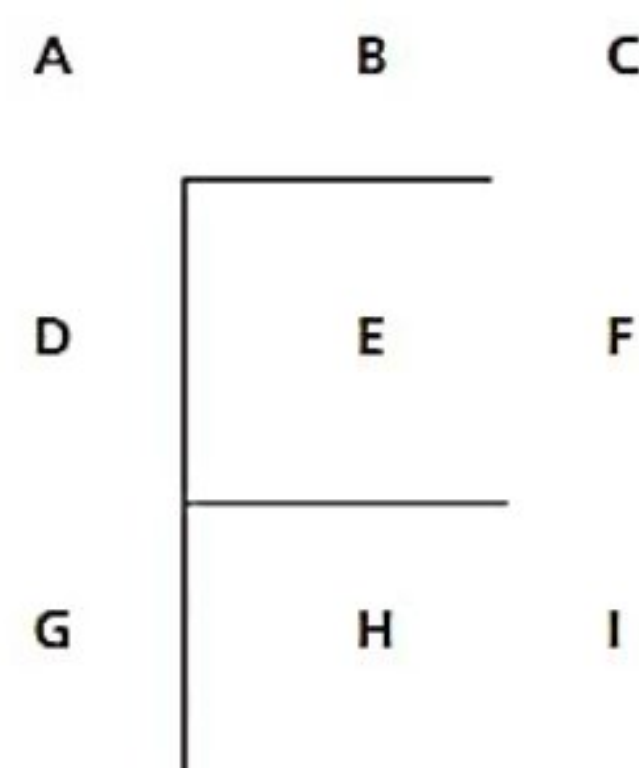


Figure 14.2: A Grid Game

suitable for environments where complete information is available. For example, the problems like object detection can be better solved using a classifier than by reinforcement learning.

Characteristics of Reinforcement Learning

1. Sequential decision making – Consider the Figure 14.3. It can be seen the path from start to goal is not done in one step. It is a sequence of decisions that leads to the goal. One wrong move may result in a failure. This is the main characteristic of reinforcement learning.
2. Delayed feedback – Often, rewards are not immediate. One must spend many moves to get final success or failure. Feedback in terms of reward is often delayed.
3. The agent actions are interdependent as any action affects the subsequent actions. For example, one wrong move of an agent may lead to failure.
4. Time related – All actions are associated with time stamps inherently as all actions are ordered as per the timeline inherently.

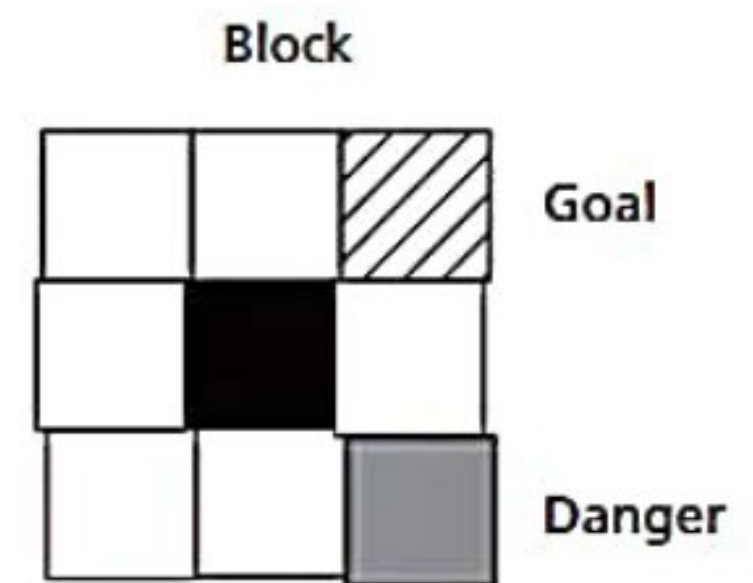


Figure 14.3: A Grid Game

Challenges of Reinforcement Learning

1. Reward design is a big challenge as in many games, as determining the rewards and its value is a challenge.
2. Absence of a model is a challenge – Games like chess have fixed board and rules. But, many games do not have any fixed environment or rules. There is no underlying model as well. So, simulation must be done to gather experience.
3. Partial observability of states – Many states are fully observable. Imagine a scenario in a weather forecasting where the uncertainty or partial observability exists as complete information about the state is simply not available.
4. Time consuming operations – More state spaces and possible actions may complicate the scenarios, resulting in more time consumption.
5. Complexity – Many games like GO are complicated with much larger board configuration and many possibilities of actions. So, labelled data is simply not available. This adds more complexity to the design of reinforcement algorithms.

Applications of Reinforcement Learning

There are many applications of RL. Some of the application domains where reinforcement learning is used are listed below:

- Industrial automation
- Resource management applications to allocate resource
- Traffic light controller to reduce congestion of traffic
- Personalized recommendation systems like news
- Bidding for advertisement

- Customized applications
- Driverless cars
- Along with deep learning games like Chess and GO
- Deep mind applications like to generate programs and images

Scan for information on 'Context of Reinforcement Learning' and 'SoftMax Method'



14.3 REINFORCEMENT LEARNING AS MACHINE LEARNING

In supervised learning, there is a supervisor but in reinforcement learning the supervisor is absent. Supervised learning learns using labelled data. It maps the input to fixed, pre-defined output. Supervised learning has a luxury of labelled data. In reinforcement learning, there is no labelled data. Instead, the labels are generated by the interaction of the agent with the environment. The agent may be a human or a computer program such as a chatbot. It can be obvious that games like Chess and GO cannot be solved using supervised learning as training dataset of these games cannot be generated. In the absence of the training set, the only option left is reinforced learning where the agent can interact with the environment and learns by trial and error.

Thus, one can observe the difficulties faced by reinforcement learning when compared with other machine learning algorithms. In supervised classifiers, the choice is one-shot, say cancer or not cancer. But in reinforcement learning, it is a sequential decision making, say finding the path from start to goal state. One must wait for more moves to see success or failure. The reason behind success and failure is found by observing the patterns over a long time and these decisions have longer consequences. For example, in a game, one wrong move may result in failure.

Reinforcement learning is different from supervised learning. The differences are listed in the following Table 14.1.

Table 14.1: Differences between Reinforcement Learning and Supervised Learning

Reinforcement Learning	Supervised Learning
No supervisor and labelled dataset initially	Presence of supervisor and labelled data
Decisions are dependent and are made sequentially	Decisions are independent of each other and based on input given in the training phase
Feedback is not instantaneous and delayed by time	Usually, the feedback is instantaneous once the model is created
Agent action affects the next input data	Dependent of initial input or the input given at start
No target values, only goal oriented	Target class is predefined by the problem
Example: Chess, GO, Atari Games	Example: Classifiers

The differences between reinforcement learning and unsupervised learning are listed in Table 14.2.

Table 14.2: Differences between Reinforcement Learning and Unsupervised Learning

Reinforcement Learning	Unsupervised Learning
Mapping from input to output is present	Not present
Get constant feedback from environment	No feedback from environment

14.4 COMPONENTS OF REINFORCEMENT LEARNING

The components of reinforcement learning are shown in Figure 14.4. These are environment, agent, actions and rewards.

Reinforcement Problems

There are two types of problems in reinforcement learning – Learning and Planning.

In learning problems, the environment is unknown and the agent learns by trial and error. The agent interacts with the environment to improve policy.

Planning is another problem where the environment is known and the agent computes with the model and improves policy.

Environment and Agent

Environment is the world where all actions take place. It is the framework, where the input, output and reward are specified. The environment describes the state or state variables or simply as state. Initially, the environment is in a state called initial state.

For example, in a car system, the maps, game rules and obstructions in the road are described in the environment.

An agent is an autonomous body that looks at the environment and takes an action. It can be any human or another computer program such as a robot or chatbot.

States and Actions

The input for reinforced learning is called a state and the output is action.

Consider the following graph in Figure 14.5 generated by two actions, up and down, by an agent.

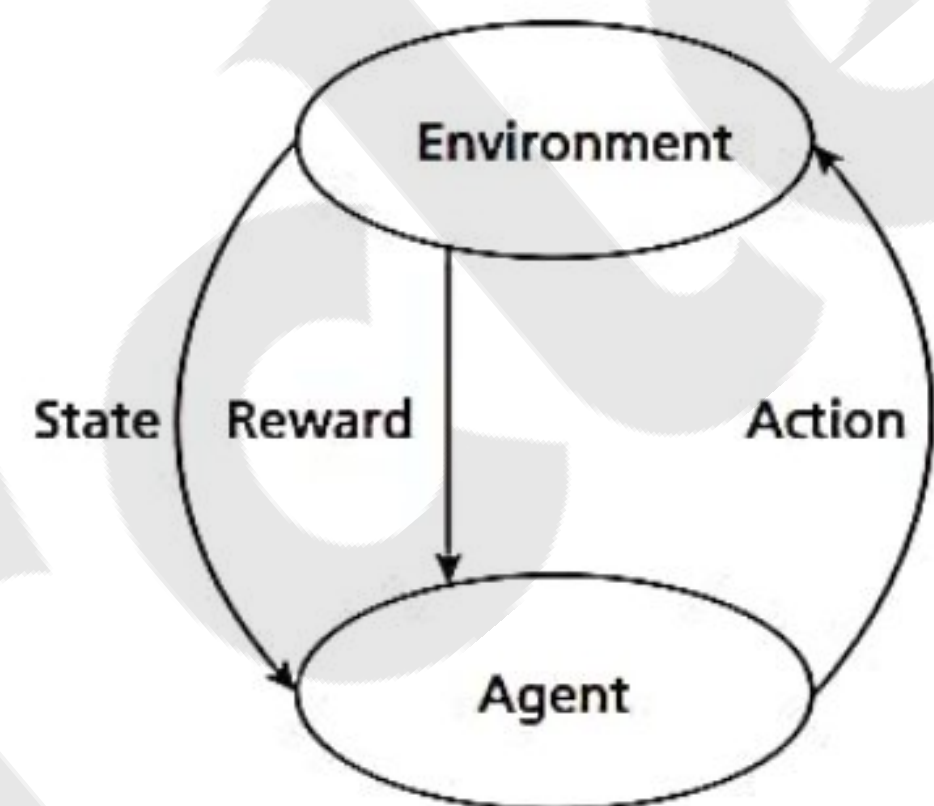


Figure 14.4: Basic Components of RL

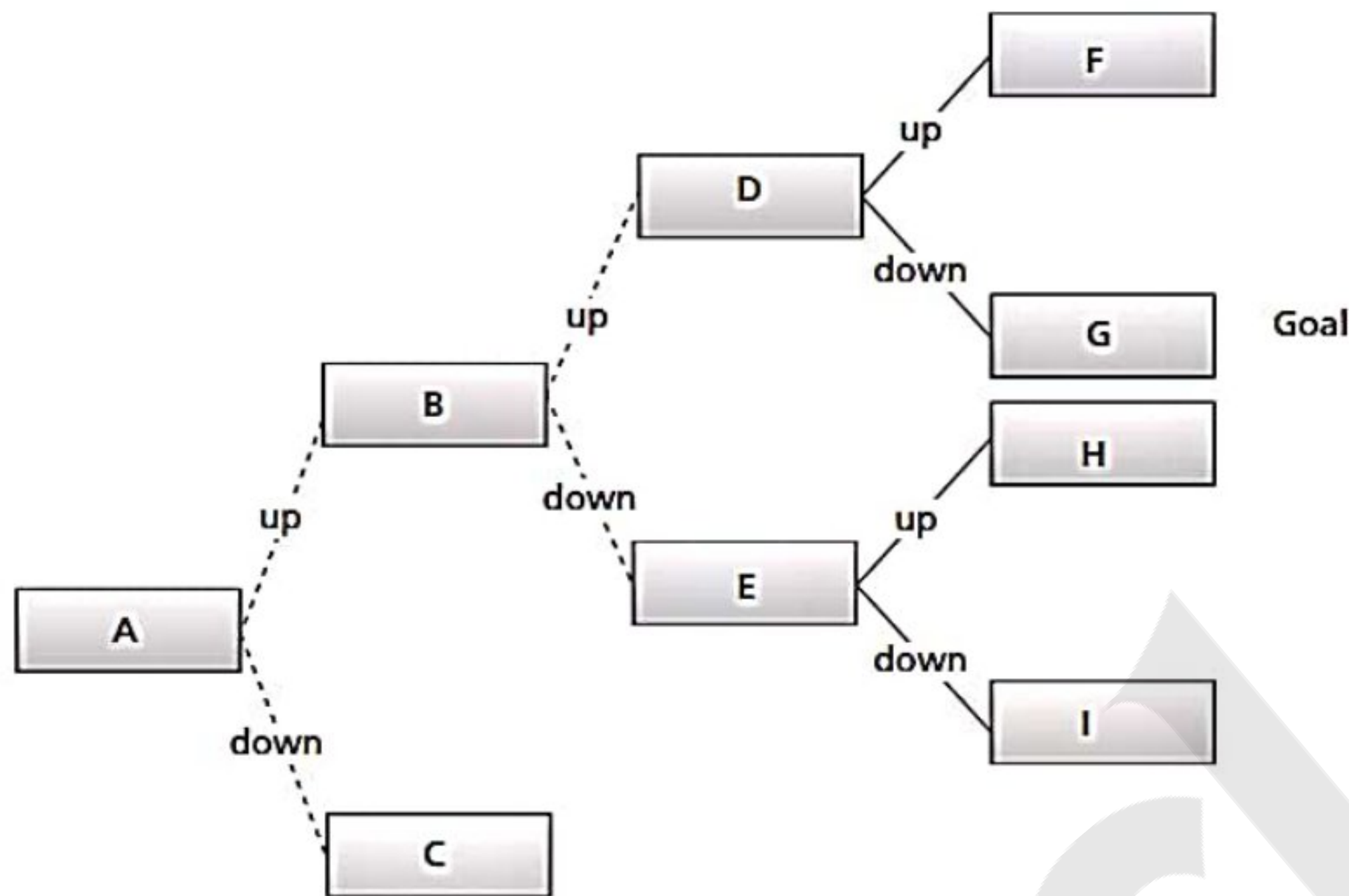


Figure 14.5: A Game: UP Action Results in Darker Line and Down Operation Results in Dotted Lines

In Figure 14.5, the states are A, B, C, D, E, F, G, H and I. These states can be cities of a shortest path problem or any other state encountered by a robot navigation. Let A be the starting state and G the goal or target state to be reached. The symbol ' S ' is used to denote the general state and ' s ' the specific state. ' s_t ' is used to denote a state at time ' t '.

Other important nodes are:

1. GOAL node – This is also known as terminal node and absorbing state. It is the goal and the agent aims to get as it is the highest discounted cumulative node.
2. Non-terminal node – All other nodes are called non-terminal nodes.
3. Start – The initial state.

Actions are transitions between states, say the path between A and B is caused by UP action. Then, the agent collects the rewards. In the game shown in Figure 14.5, the actions are UP and DOWN only. Symbol A is used to denote the general actions – UP and DOWN. The symbol ' a ' is used to specify the specific action and ' a_t ' is used to denote the action at time ' t '.

'Episodes' is the number of steps necessary to reach the goal state from start state. For example, in Figure 14.5, the aim is to find optimal path between A and G. There are two types of episodes.

One is called episodic and another is called continuous. In episodic tasks, there is a well-defined starting state and end state. End state is called as terminal state or goal state. An episode is one which consists of states, actions, and rewards. For example, all the paths from A to G, such as

A → B → D → F → G

A → B → E → G

are all examples of episodes. An episode is one that starts from a starting state and reaches the goal state. So, a successful episode would be E-F-C-B-A-D-G.

Some times, there may not be any goal node. A continuous task has no terminal state, that is, no designated goal states.

Policies

The role of algorithms is called a policy in reinforced learning and policy maps states to actions. In short, a policy takes the input states, processes it and suggests an action. Policies characterize the agent behaviour function. There are two types of policies – deterministic and stochastic.

The deterministic policy function is given as:

$$a = \pi(s) \quad (14.1)$$

A deterministic policy states that a given state always returns the same action, i.e., premapped.

The stochastic policy function is given as:

$$\pi(a | s) = \text{Pr}[a_t = a | s_t = s] \quad (14.2)$$

It is the probability of taking an action 'a' while in state 's'. The aim of the agent is to learn policy function so that the best actions are selected.

Given multiple policies, how to decide a better policy? There are many policies. A better policy is the one that has better cumulative expected reward over other policies. More formally, the value of the state is higher for the best action if the policy is optimal.

Rewards

The system performance is measured by reward system. A reward is something that the environment produces to the agent. In short, it maps a set of states to a real number. The symbol 'r' is used for reward, which depends on current state 's', action 'a' and next state 's'.

Immediate Reward The environment immediately delivers a reward after the execution of action and the total reward is associated with the episode. This is the sum of all rewards that come from the first move to the last move. The total reward is also called as return or goal.

It can be observed that every game has states. It then selects an action and transits to another state. As a result of the transition, it collects the rewards too. This continues till the terminal state. The return is given as:

$$G_t = r_{t+1} + r_{t+2} + r_{t+3} + \dots + r_T \quad (14.3)$$

Here, G_t is the return and r_T is the terminal or final state.

Long-term Reward Unlike immediate rewards, long-time rewards are given after the end like the objective of the game.

The total future reward is the sum of the current move reward and all other moves' rewards till the end of the episode. For example, if there are 9 moves in an episode, the total future reward of the third move will be the sum of reward of that move and rewards of move 4 to move 9 present in that episode.

More often, the iteration can be without terminal state too. In other words, it is a continuous task. To compute future rewards or utility, one can introduce a factor γ called discount factor:

$$\begin{aligned} G_t &= \gamma r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots \\ &= \sum_{j=0}^{\infty} \gamma^j r_{t+j+1} \end{aligned} \quad (14.4)$$

G_t is called return or utility or total reward. It is a value that indicates the closeness of the goal state.

Its value ranges from 0 to 1. It gives more value for the immediate return. The unpredictability can be tackled by discount factor, which is the sum of the immediate reward and all other subsequent rewards multiplied by a factor of γ .

When γ is 1, the discount factor is close to the total reward. When γ is zero, only immediate reward matters and virtually all future actions are not taken into account. Often, the value of γ is chosen as 0.8 or 0.9.

Example 14.1: Let us assume that at the fifth step, the reward is +1. If the discounted factor γ is 0.7, what is the discounted reward or utility? Assume that the previous steps' rewards till the fifth step are zero.

Solution: The reward at 5th step is 1. Therefore, the discounted reward using the Eq. (14.4) is given as:

$$(0.7)^5 \times 1 = 0.16807$$

Broadly stated, reinforcement algorithms are classified as model-based and model-free methods based on the availability of models for the given problem. Certain real-world problems can be solved by constructing models using Markov decision process. Let us discuss about it now.

14.5 MARKOV DECISION PROCESS

Markov Decision Process (MDP) is an extension of Markov Chain. What is a Markov chain? A Markov chain is a stochastic process satisfying Markov property. It is a sequence of random variables $X_0, X_1, \dots, X_n$ satisfying conditional probability.

A Markov chain built on two states is shown in Figure 14.6. Let us assume that there are two universities. Eighty percent students of university A leave for university B for getting a masters' degree and only 20% remain in the same university to study further. Similarly, 60% of students of university B move to university A and 40% remain to study in the same university. The Markov chain for this is given as follows:

It can be observed that there are two states. The transition between state A and state B is an edge with probability. For example, the transition between states A and B is 0.8. A transition matrix P of a Markov chain at time t is a matrix that has probability of transition that exists between two states. The transition matrix at time t is given as follows:

$$(P_t)_{i,j} = Pr(X_{t+1}=j | X_t=i)$$

The transition matrix for the Markov chain is given as follows:

$$P = \begin{pmatrix} 0.2 & 0.8 \\ 0.4 & 0.6 \end{pmatrix}$$

The sum of rows of P is 1. In other words, a row of the matrix is a probability vector.

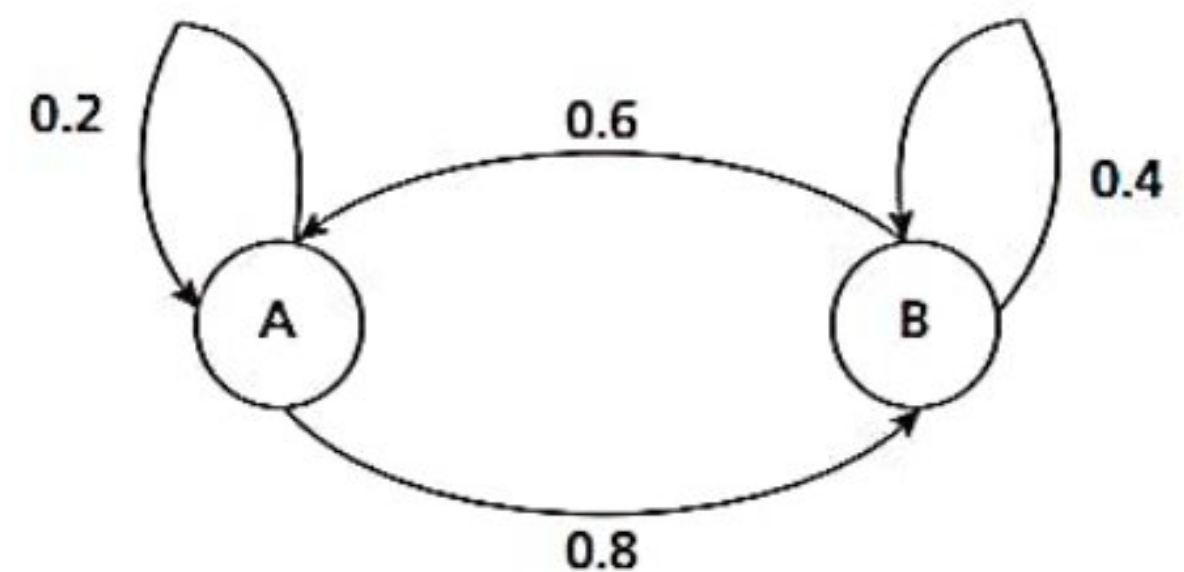


Figure 14.6: Markov Chain

If u be the probability vector that represents the initial state of the Markov chain, the probability of the chain in state ' s ' after ' n ' steps is the i^{th} entry of the vector:

$$u_{n+1} = u_n \times P \quad (14.5)$$

The next state is computed by multiplying u and P .

Example 14.2: Assume the universities' hold on a town is initially 60% and 40%. So, what would be the prediction after one month, that is, the hold of university after an year. After two years, will the hold remain same or change? Assume the rest of the probability transition information from Figure 14.6.

Solution: The starting distribution is $u_0 = (60\%, 40\%) = (0.6, 0.4)$. The Markov chain is shown in Figure 14.6 and the transition matrix is given as:

$$P = \begin{pmatrix} 0.2 & 0.8 \\ 0.4 & 0.6 \end{pmatrix}$$

The prediction after a year as in Eq. (14.5), $u_1 = u_0 \times P$, will be:

$$\begin{aligned} &= (0.6 \ 0.4) \begin{pmatrix} 0.2 & 0.8 \\ 0.4 & 0.6 \end{pmatrix} \\ &= \begin{pmatrix} 0.28 \\ 0.72 \end{pmatrix} \end{aligned}$$

The hold of the universities will be 28% and 72%. This is obvious because 80% of the students shift from the university A to B.

The second-year prediction $u_2 = u_1 \times P$ would be:

$$\begin{aligned} &= (0.28 \ 0.72) \begin{pmatrix} 0.2 & 0.8 \\ 0.4 & 0.6 \end{pmatrix} \\ &= \begin{pmatrix} 0.344 \\ 0.656 \end{pmatrix} \end{aligned}$$

It can be observed that the hold on the student by the universities changes.

If rewards are added to markov chain, the markov decision process is obtained where the edges are associated with probability as well as reward.

As discussed earlier, MDP is an extension of Markov Chain. The components of MDP are:

1. A set of states
2. A set of actions
3. Reward Function R
4. Policy π
5. Value V

The simplest structure of MDP is Markov chain where the environment is modelled as a graph. The nodes of the graph represent states and edges represent transitions. Each edge is associated with a static probability.

Markov assumption holds good for reinforcement learning. Markovian property states that the probability of reaching a state s_t and getting the reward r_t depends solely on the preceding states s_{t-1} and r_{t-1} . All other previous states are not important. This is called Markov assumption.

A state is Markov if and only if:

$$p[s_{t+1} / s_t] = p[s_{t+1} / s_1, s_2, \dots, s_t] \quad (14.6)$$

Here, s' and s are new and old states and ' a ' is the action, and ' r ' is the reward.

MDP performs the following actions:

- Observes the current state s_t
- Performs the action a_t
- Gets the reward $r(s_t, a_t)$
- Enters a new state s_{t+1}

The aim of the agent is always to maximize the total rewards accumulated over time. The movement of one state to another is called a transition. The agent manipulates the environment from one state to another.

The state transition probability is given as follows:

$$P_{ss'}^a = \{s_{t+1} = s' \mid s_t = s, a_t = a\} \quad (14.7)$$

It indicates the moving from states to s' after taking an action ' a '. In other words, given a state ' s ', it is the probability of the occurrence of next state s' . One can also represent it as a matrix called state transition matrix where each row indicates the transition probability from one state to another.

The expected value of the reward is given as:

$$R_{ss'}^a = E\{r_{t+1} \mid s_t = s, a_t = a\} \quad (14.8)$$

Training and Testing of RL Systems

Once the MDP is modelled, the next stage is called training and inference. The agent tries to accomplish a goal repeatedly and modify its parameter over time. This is called learning. Once the trained model is available, it is put to use and this process is called inference. At deployment level, training and inference go together and when environment is changed, training is done again. This is to improve the system.

14.6 MULTI-ARM BANDIT PROBLEM AND REINFORCEMENT PROBLEM TYPES

Reinforcement learning uses trial and error to learn a series of actions that maximize the total reward. There are two sub-problems in the reinforcement problem.

1. The main problem is the prediction of total reward or return. This is called prediction, policy evaluation or value estimation. It requires a formulation of a function called state-value function. The problem of estimating the state value function can be done using temporal differencing or TD-Learning.

2. The second problem is finding actions that maximize the returns. It is called policy improvement. Both these things can be combined as policy iteration which alternatively uses the above to find optimal policy.

Let us introduce a problem that is encountered in reinforcement learning very often called multi-arm (or N-arm) bandit problem. A 5-armed slot machine is shown in Figure 14.7.

Assume a robotic arm in a hypothetical casino that when activated through a lever returns money in the range, say, \$1 to \$10. A multi-arm bandit problem can have N levers. The important issue is that the arm randomly returns the money in the range \$1–\$10. This is called a reward. Given a limited chance, the aim of the user is to get as much money as possible.

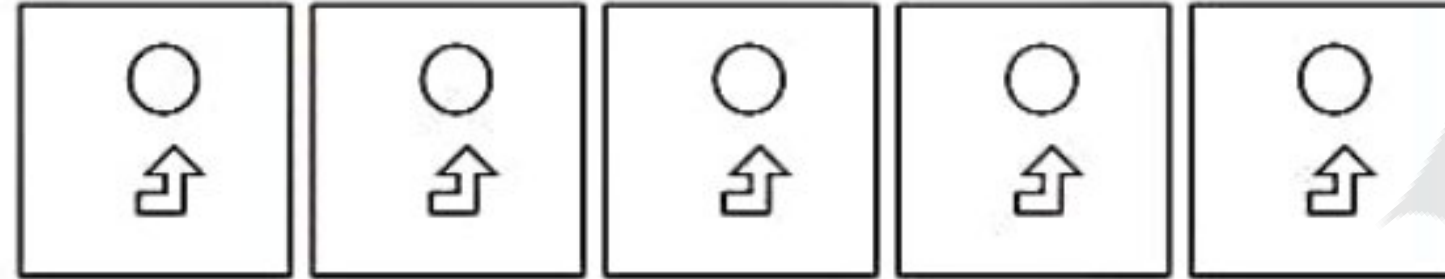


Figure 14.7: Arm Slot Machines

The point is which lever is the best one to activate to get maximum profit. A possible answer to this question is: the lever that returns average return more than other levers is the best slot machine. Let us formalize the problem: Given k number of chances to access N-arm slot machine, with actions of pulling the lever with associated rewards, $r_1, r_2, \dots, r_k$, the expected reward given is given as:

$$Q_k(a) = \frac{r_1 + r_2 + \dots + r_k}{k} \quad (14.9)$$

This function is called action-value function or Q function. This indicates the value of taking a particular action ' a '.

$$Q_* = \arg \max_a Q_k(a_i) \quad (14.10)$$

The best action (highest Q_* value) is the action that returns the highest average return and is the indicator of the action quality.

Example 14.3: Assume that a slot machine that is chosen five times returns rewards, 1, 2, 0, 7 and 9. What is the quality of this action?

Solution: The quality of this action as given in Eq. (14.9) is:

$$Q_k = \frac{1 + 2 + 0 + 7 + 9}{5} = \frac{19}{5} = 3.8$$

In order to avoid storing these rewards, use a recursive formula to compute the average reward incrementally as:

$$Q_{k+1}(a) = Q_k(a) + \frac{1}{k+1} [r_{k+1} - Q_k(a)]$$

Next, let us discuss the selection of actions called selection policy.

Exploration vs Exploitation and Selection Policies

The selection of actions may be based on the concept of exploration and exploitation.

Exploration The method explores all actions even if it results in sub-optimal decisions. It is considered to be an adventurous method. In games, it works by exploring all actions and observing rewards.

Exploitation It is a naïve approach that uses current knowledge and keeps exercising those options repeatedly. Its aim is to choose only one action based on experience and always get the same reward. This is not adventurous and is mostly suboptimal.

There must be a balance between exploration and exploitation for selection. Based on these principles, selection policies can be designed.

Greedy Method

In the absence of any knowledge, picking the best action as of now is called greedy method. This is based on the principle of exploitation. The information about greedy approach is given in Box 14.1.

In a multi-arm bandit problem, initially all levers are tried. The agent notes down all the rewards that are returned by these levers. By learning the lever that returns the maximum rewards, the agent then uses it repeatedly to reap maximum reward. Say, for example, if the fifth lever returns more money, then the same lever is pressed again and again to get maximum reward. This is called greedy approach and hence there is no exploration part. For example, eight levers might have returned more, but one may never know due to the experience with the fifth lever.

Box 14.1: Greedy Approach

Greedy approach is an algorithm design technique for solving optimization problems. An optimization problem is one that has an objective function that needs to be maximized or minimized. Greedy approach generates all feasible candidate solutions. Out of all these solutions, the optimal solutions are picked by using local optimality (best solutions as of now) that maximize or minimize the objective function based on the problem requirement.

The Greedy approach solves problems that have optimal substructure. It means an optimal solution within itself contains optimal solution for its subproblems. For example, let us assume the path between A and C that passes through B is optimal. If so, then the paths from A to B and B to C are optimal with this criterion. The second property is called greedy choice property, where a problem can be solved by combining the solutions of subproblems to get a global optimal solution. This requires the greedy approach to generate all candidate solutions, select the best solution and do a feasibility check.

To illustrate the concept, let us consider a simple problem called coin change or cashier problem. For example, if anyone asks 100 rupees, certainly he would not prefer to receive 100 one-rupee coins. In other words, one would like to receive return in highest denominations. Formally stated, given denominations of, say 1, 5, 10-rupee coins, the problem is to minimize the number of coins. The coin change problem can be framed as optimization problem as:

Objective function: To minimize the number of coins

Candidate solutions: Many, that is, say 17 one-rupee coins, one 10, one 5 and two 1-rupee coins, three 5-rupee coins and two 1-rupee coins. There may be many candidate solutions.

(Continued)

Feasibility check and solution: The sum of all coins should equal 17 rupees, not more, not less. This is the solution check. So, three five-rupee coins cannot be given as it exceeds the number and is not feasible. So, what is the optimal solution?

Selection procedure is greedy principle of 'largest denomination'. So, the answer for the problem is that the highest denomination is 10. Therefore, it is issued. Therefore, the solution is {10}. Deduct 10 from 17. It is 7. Then, look for the highest denomination, that is, 5. So, the solution is {10, 5}. Deduct 5 from 7 to get 2. The largest denomination is 1. Therefore, the solution is {10, 5, 1, 1}. The number of coins is 4.

While greedy approaches normally work, and hence heuristics, occasionally they fail due to the short-term strategy.

ϵ – Greedy Method

This method combines both exploration and exploitation. In ϵ – greedy, ϵ value ranges from 0 to 1 (refer to Algorithm 14.1).

Algorithm 14.1: ϵ - Greedy

Step 1: Generate a random number p in the range 0 – 1.

Step 2: If $p \geq \epsilon$, the algorithm chooses the action with highest reward value.

Step 3: If $p < \epsilon$, actions are chosen randomly.

Step 4: Update the value of action and repeat for all actions.

With probability ϵ , the action a is chosen randomly, but rest of the time with probability $1 - \epsilon$, the best actions based on greedy approach are selected. Thus, there is a balance of exploration as well as exploitation. For example, if ϵ is 0.2, then 80% best actions are chosen and 20% random actions are chosen.

Reinforcement Agent Types

An agent can be characterized using these methods:

1. Value-based approaches
2. Policy-based approaches
3. Hybrid methods
4. Model-based methods
5. Model free methods

Value-based Approaches The aim of value-based approaches is to optimize $V(s)$. Here, $V(s)$ is a function that returns the maximum expected future reward the agent will get at each state discounted by a discount factor γ . This value is then used to select the state for the agent. A large value is chosen by the agent to pick the state.

Policy-based Approach The aim is to find an optimal policy. A policy π is a strategy. A policy is a function that maps a state to a probability distribution of all possible actions in that strategy. It aims to find the optimal policy to map each state to best action.

Actor Critics Methods These are hybrid methods combining both value-based and policy-based methods.

Model-based Approach First, the model is constructed for the environment as it is available. Markov Decision process is one way of modelling the given problem.

Model-free Methods Here, models are not available and hence some methods are used to simulate the model virtually. The techniques like temporal differencing and monte-carlo methods are generally used to simulate such models. These techniques are discussed in section 14.8.

Reinforcement Algorithm Selection

Reinforcement agent algorithms are decided based on the criteria of availability of models, environment and nature of updates. The agent algorithm selections are shown in Table 14.3.

Table 14.3: Algorithm Selection

Model	Environment	On/off	Algorithms Used
Present	Static	Off	Dynamic Programming
Absent	Dynamic	On	SARSA
Absent	Dynamic	-	Temporal Learning, Monte Carlo Methods
Absent	Dynamic	Off	Q-Learning

14.7 MODEL-BASED LEARNING (PASSIVE LEARNING)

Passive Learning means model-based environment, which is based on known environment. By known environment, it means that for any state, the next state and action, the probability distribution is known. Markov Decision Process (MDP) and dynamic programming are excellent tools for solving reinforcement problems.

The mathematical framework required for passive learning is provided by Markov Decision Process. These model-based reinforcement problems can be solved using dynamic programming (refer to Box 14.2) after the construction of the model using MDP.

Box 14.2: Dynamic Programming

Dynamic Programming is an algorithmic technique to solve optimization problems. Richard Bellman is credited with the design of dynamic programming approaches.

Dynamic programming is used to solve the optimization problem by dividing that into a set of overlapping subproblems. Then, the solution of the problem is created as a recursive formulation. The strategy is to solve the subproblems and store in a table. Then, the original problem is solved by reusing the solutions that are already stored in the table. The effectiveness of dynamic programming approach is due to the concept of 'reuse' instead of recomputing the solutions for

(Continued)

problems repeatedly. This recursive equation that characterizes the problem is called Bellman equation named after Richard Bellman.

Thus, the approach of dynamic programming can be summarized as follows:

Step 1: Split the problem into a number of subproblems. The subproblems are interrelated and are overlapping in nature.

Step 2: A table is created. Whenever a subproblem is solved, its solution is stored in a table.

Step 3: The solutions of the subproblems are combined in a bottom-up fashion to find solution for the original problem.

The main principle of dynamic programming is 'Principle of Optimality' that states that optimal solution of a multi-stage problem is feasible if and only if its subsequences are optimal. The recursive formulation of the problem is must. Thus, the characteristics of a dynamic programming is overlapping subproblems, storage in a table (memoization technique – a corrupted word of memorization) and finally optimality substructure as greedy algorithms).

Dynamic programming requires two essential properties, optimal substructure and overlapping problems. MDP satisfies both the properties of dynamic programming.

Therefore, dynamic programming is an offline algorithm as it needs to wait till the end of episode to update values and actions. Compared to dynamic programming, reinforced learning uses online updates of state values and actions during the course of the episode.

Scan for Additional Information on 'Dynamic Programming'



In reinforcement learning, the focus is to take action a to transit from the current state to the end state. The aim is to get rewards for the actions. They can be positive or negative.

The objective is to maximize the rewards by choosing the optimal policy.

$$\text{Max } E(r_t | r, s_t) \text{ for all possible values of 's' at time 't'}$$

There are multiple courses of actions for a given state. The agent can prefer any course of action. So, the behaviour of the agent needs to be understood. The algorithm to do that is called a policy. A policy is the distribution of all actions with the probability of choosing these actions. Different actions can have different rewards. It is necessary to find the total reward. This is possible only if the reward is quantified. It is done using value functions.

Value Notation

Value functions summarize all possible future scenarios by averaging over specific returns for a given policy π . In other words, it is a prediction of future reward. It computes the expected sum of all future rewards for a given state ' s ' for a policy π . It is denoted as $v(s)$. It indicates the quality of the state based on a long-term strategy.

For example, if there are two states with values 0.2 and 0.9, then the state associated with 0.9 is a better state to be in. Value functions can be either for states or state-action pair. Value function for state is called state-value function. Value function for state-action pair is called state-action function.

State Value Notation Let v represent a value of the state. The state-value function is defined as follows:

$$\begin{aligned} v_{\pi}(s) &= E_{\pi}(G_t | s_t = s) \\ &= E_{\pi} \left\{ \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} | s_t = s \right\} \end{aligned} \quad (14.11)$$

It implies that state-value function of an MDP is the expected return from state ' s ' for a policy. Here, $V_{\pi}(s)$ adds up all expected rewards, possibly discounted rewards. It also indicates how good the state is! The subscript π indicates that the value depends on policy π .

The value function quantifies the amount of reward for an action or state. It gives the expected sum of future rewards for a given state under policy π .

The optimal state-value function is denoted as:

$$v_*(s) = \max_{\pi} v_{\pi}(s) \quad (14.12)$$

In other words, it is the maximum value function over all policies.

Dynamic programming is used to solve this problem. As discussed in Box 14.2, dynamic programming requires the recursive formulation of the given problem. This recursive formulation for solving the state-value problem is called Bellman equation and is given as:

$$v_{\pi}(s) = \sum_a \pi(s, a) \sum_{s'} P_{ss'}^a + [R_{ss'}^a + \gamma v_{\pi}(s')] \quad (14.13)$$

Action Value Function Apart from value function, one more function called Q function is introduced. This function returns a real value. It is a function that indicates the total reward by the agent in state s and picks an action ' a ' and follows a policy π . It indicates how good the action is in the state. It is denoted as follows:

$$Q_{\pi}(s, a) = E_{\pi}[G_t | s_t = s, a_t = a] \quad (14.14)$$

This shows the policy π as the probability of choosing an action ' a ' when in state ' s '.

v_* is the maximum total reward when starting from state ' s '. It will be a maximum of $Q_*(s, a)$ overall possible actions. Therefore,

$$v_* = \max_a Q_*(s, a) \text{ for all state}$$

If Q is known, then the best action to take optimally can be given as:

$$Q_*(s, a) = \max_{\pi} Q_{\pi}(s, a)$$

Also, one can write Q_* in terms of v_* as follows:

$$Q_{\pi}(s, a) = E\{r_{t+1} + \gamma v_{\pi}(s_{t+1}) | s_t = s, a_t = a\}$$

Algorithms for Solving Reinforcement Problems using Conventional Methods

After defining the state-value and action-value functions, one can define the algorithms to solve reinforcement problems. There are two types of algorithms:

1. Value Iteration
2. Policy Iteration

Value Iteration Value iteration estimates v_* in term of steps as:

$$v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_*$$

The algorithm for value iteration is given as follows:

Algorithm 14.2: Value Iteration

Step 1: At each iteration $k + 1$, for all states $s \in S$.

Step 2: Update $v_{k+1}(s)$ from $v_k(s')$ where s' is the successor state of s .

$$\text{Step 3: } v_{k+1}(s) = \max_{a \in A} \sum_{s'} P_{ss'}^a [R_{ss'}^a + \gamma v_k(s')] \quad (14.15)$$

Slowly, this converges to $v_*(s)$ and finally the policy is:

$$\pi(s) = \arg \max_a \sum_{s'} P_{ss'}^a [R_{ss'}^a + \gamma v_k(s')] \quad (14.16)$$

Policy Iteration The second algorithm is policy iteration. The task of finding an optimal policy consists of two steps:

- Policy Evaluation
- Policy Improvement

Policy Evaluation

Initially, for a given policy, the algorithm starts with value v_1 function as zero. There is no reward. The Bellman equation is used to obtain value of v_2 . Then, the value of v_3 is obtained from v_2 . This process is continued till the optimal policy v_* is obtained. In other words, at each iteration, all the state values are updated using the previous iteration value.

Policy evaluation estimates v_* in terms of steps: $v_1 \rightarrow v_2 \rightarrow v_3 \rightarrow \dots \rightarrow v_*$ and policy improvement is possible if the new policy $\pi' > \pi$. The improvement is done using greedy approach.

The algorithm for policy evaluation is given as follows:

Algorithm 14.3: Policy Evaluation

While (True)

 Initialize $v(s) = 0$ for all states s

$\Delta = 0$

 For each state, $s \in S$ do

$v_{old}(s) = v(s)$

 Compute new $v_{new}(s)$ function:

$$v_{new}(s) = \sum_a \pi(s, a) \sum_{s'} P_{ss'}^a [R_{ss'}^a + \gamma v(s)] \quad (14.17)$$

(Continued)

Repeat until $\max(\Delta, |v_{new}(s) - v_{old}(s)|) < \epsilon$ then break

Return

Finally, the policy converges to v_* .

Policy Improvement The policy improvement can be done as follows:

1. Evaluate the policy π with policy evaluation.
2. Solve the Bellman equations for the given policy to get v_π .
3. Improve the policy by applying the greedy approach for policy v_π so that the policy becomes π' .
4. Repeat the process till v_π policy converges to optimal policy v_* .

The algorithm is given as follows:

Algorithm 14.4: Policy Improvement

For each $s \in S$

Old = $\pi(s)$

$$\pi'(s) = \arg \max_a \sum_{s'} P_{ss'}^a + [R_{ss'}^a + \gamma v(s')] \quad (14.18)$$

If old $\neq \pi(s)$, then GOTO policy evaluation else EXIT.

Scan for 'Additional Examples'



14.8 MODEL FREE METHODS

In model free methods, there is no complete knowledge of the environment. So, how to determine the rewards then?

The simplest formulation is the method called episodic formulation. Recollect from section 14.2 that an episode is the path from start to the goal node. For example, the shortest path between state A to state G in Figure 14.5 is called an episode.

The simplest way to determine rewards is to play the game. If the game ends in win, then all actions that are encountered in the episode are given a reward, say +1. If the game is lost, then all actions are given a value of -1. The only problem is that the game may be lost due to the last bad move while all other actions may be good.

The second method is the continuous formulation. Here, the reward is determined after the action. For example, in multi-arm bandit problem, the immediate reward of \$1 – \$10 can be an example of continuous formulation. If the actual value is not known, then it can be estimated.

The third formulation is the discounted returns. The discounted return computed as the long-term reward can be one solution.

Some of these techniques are used in these model-free techniques such as:

1. Monte carlo (MC) methods
2. Temporal Difference Learning

One of the simplest ways of simulating a model is by using Monte-Carlo methods. This is discussed in the subsequent section.

14.8.1 Monte-Carlo Methods

Monte Carlo (MC) methods do not assume any model. Therefore, complete knowledge of the environment is not available. This is similar to human and animal experience where the knowledge is gained by experience, and experience is gained from the interaction with the environment. MC uses the methods of experience as well as simulation to gain knowledge of the environment.

The experience is divided into many episodes. An episode, as discussed earlier, is a sequence of states from starting state to goal state.

MC makes the following assumptions:

1. All episodes should terminate. No matter from where we start, the episode should end.
2. MC methods use value-action functions. These values are computed only after the completion of the episode. So, MC methods are incremental methods.

The MC method collects the reward at the end of the episode. Then, it is used to find maximum expected future reward. This method does not use expected return. Instead, it uses the empirical mean as return. The total return over many episodes divided by the number of episodes gives the average, alternative way to count every time a state is encountered and reward until termination.

Since the problem is non-stationary, the value function is computed for fixed policy. Then, the policy is revised using dynamic programming.

The mean value is computed as:

$$v(s_t) = \frac{1}{n(s_t)} \sum_{i=1}^{n(s_t)} G_t^i \quad (14.19)$$

Thus, the incremental Monte-Carlo update at the end of the episode using the approach - incremental every-visit Monte-Carlo is given as:

$$v(s_t) = v(s_t) + \alpha(G_t - v(s_t)) \quad (14.20)$$

Here, α is called learning rate. The algorithm for MC methods is as follows:

Algorithm 14.5: MC

- Step 1: For a policy π , select state s .
- Step 2: Generate an episode.
- Step 3: Compute average returns for every first occurrence of state s in an episode.
- Step 4: Update the returns and start a new episode. Repeat for all episodes.

14.8.2 Temporal Difference Learning

Temporal Difference (TD) learning is an alternative of Monte Carlo method. It is model free. It learns from environment based on experience and interaction as MC methods are based on the principle of bootstrapping. TD is called bootstrapping method because the update is based on the existing estimate and future reward. The difference is the updated new estimate. In TD method, in every step an estimate of the final reward is calculated at each state. Then, immediately the state-action is updated for every step.

Unlike Monte Carlo method that computes the rewards at the end of the episode, TD Learning computes the rewards at every step. The difference is listed in the following Table 14.4.

Table 14.4: Differences Between Monte-Carlo Method and TD Learning Method

Monte-Carlo Method	Temporal Difference Method
It is easy to understand and does not exploit Markov principle	Exploits Markov principle
This requires termination and episode	Does not require termination

The temporal differencing (TD) target is given as:

$$r_{t+1} + \gamma v(s_{t+1}) \quad (14.21)$$

In TD(0), the update towards the estimated return is given as:

$$v(s_t) = v(s_t) + \alpha (G_t - v(s_t)) \quad (14.22)$$

Here, α is constant parameter and r_t is the reward after time ' t ', and s_t is the state visited at time ' t '.

TD Learning can be accelerated through eligibility traces as future rewards may be done for one to many nodes. As soon as it receives an input, it uses a table so that the previous prediction is updated. Eligibility traces provide an alternative short-term memory that all previous predictions are noted in the table. This leads to a family of algorithms as $TD(\lambda)$. Here, λ is called decay parameter whose value ranges between 0 and 1. If it is 0, then only previous prediction is updated. If it is 1, then all the predictions are updated.

The algorithm for TD learning is given as follows.

Algorithm 14.6: TD-Learning

1. Initialize policy π .
2. Set $v(s) = 0$ for all states $s \in S$.
3. Repeat for each episode.
4. Initialize s .
5. Repeat for each step of episode.
6. Choose action ' a ' and execute it. Observe the new state and new reward.
7. Update value using Bellman Eq. (14.22).
8. Update as s' .
9. Until s is terminal state.

14.9 Q-LEARNING

Q-Learning is an off-policy method because Q is updated based on policies that are implicit to Q and is better guaranteed for maximum returns. Q indicates Quality. What is Q -value? $Q(s, a)$ is a numerical value assigned to a state-action pair. It means a value of the action that is performed in state ' s '.

Here, the main objective is to find Q -Value? Q -value is nothing but the immediate reward and other rewards that are yet to come, known as total return reward. To compute the total return reward, these information are necessary.

1. Starting state
2. Action
3. Reward
4. New state

Initially, a table called Q -table is constructed and filled with initial random values. Q -learning involves two policies - learning policy and update policy. Q -learning is done by methods like greedy, softmax or softmax plus.

The agent's next move will be the action where rewards are high. Alternatively, it would be the cell that has the highest Q -value. As the moves are determined by Q -values, the computation and updation of Q -values are important for Q -learning. Then comes the Q -learning update policy. The updation of Q -values is done using Temporal-Differencing method that is mentioned in section 14.8.2. This updation is made by blending the new and old values. Blending is done by α . When α is zero, the value does not change at all. When α is one, the next value replaces the old value. Here, α is known as the learning rate. The discount factor (γ) is also used for update. Blending is done using temporal difference learning.

The updation procedure of Q -value is given as follows:

1. Perform any random action on state s_t
2. Get a new state, s_{t+1}
3. Get the award $r(s_t, a_t)$

The temporal difference at time ' t ' is done using the update:

$$TD_t(s_t, a_t) = r(s_t, a_t) + \gamma \max_a (Q(s_{t+1}, a)) - Q(s_t, a_t) \quad (14.23)$$

Here, $r(s_t, a_t)$ is the reward obtained by performing action a_t and $Q(s_{t+1}, a)$ is the estimate of the best action at state s_{t+1} i.e., s' and $Q(s_t, a_t)$ is the Q -value of the action a_t at state s_t . Here, the best action performed at future states discounted by discount factor (γ) is $\gamma \max_a (Q(s_{t+1}, a)) - Q(s_t, a_t)$.

If TD is high, then it is a 'surprise factor' and denotes the highest reward. If TD is less, it represents the 'frustration' factor and denotes the lesser reward. In other words, TD is a sort of 'reward'. It is initially high and slowly gets minimized as it reaches the end of the training, that is, when it reaches the final goal.

The update is carried out using the Temporal difference learning (TD) and Bellman equation. The Bellman equation that is used to update the value is given as follows:

$$Q_t(s_t, a_t) = Q_t(s_t, a_t) + \alpha TD_t(s_t, a_t) \quad (14.24)$$

The Q -Learning algorithm is given as follows:

Algorithm 14.7: Q-Learning

1. Set $Q(s_t, a) = 0$ where s_t is the terminal node in an episode.
2. For all states s and action a , set Q Value = 0.
3. Repeat.
4. Select s_t randomly.
5. Choose action a_t from Q using ϵ -greedy.
6. Perform action a_t such that $r(s_t, a_t) > 0$ and reach the next state.
7. Update the action-value function using TD using Eq. (14.24) and Bellman equation:

$$Q_t(s_t, a_t) = Q_{t-1}(s_t, a_t) + \alpha \times TD_t(s_t, a_t) \quad (14.25)$$

Until the terminal state s is reached.

Here, α is the learning rate. It is between 0 and 1. If α is zero, then nothing is updated and there is no learning at all. When setting it to a higher value such as 0.9, the learning happens fast. γ is the discount factor. It is in the range 0 – 1. It should be less so that the algorithm can converge.

The inference is simple, the best action is the maximum of Q function as follows:

$$a_t = \arg \max_a (Q(s_t, a))$$

14.10 SARSA Learning

SARSA is based on episodic formulation and discounted factor. Its name is derived from the function $Q(s, a, r, s', a')$, that is, ' r ' is the reward and $s'(s_{t+1})$ and $a'(a_{t+1})$ are the new states and actions, respectively. It selects a state ' s ' and actions ' a ' are chosen based on the ϵ -greedy approach.

SARSA is an on-policy method because each action a_t is based on Q and Q is updated based on the actions taken. SARSA converges to optimal policy and with the action-value function provided all state-action pairs are visited infinitely often.

The major difference between SARSA and Q-learning is that not necessarily the maximum reward is used to update Q value. Instead, it is based on estimates. Thus, SARSA is an online method.

SARSA is based on estimated optimal action:

$$a_{t+1}^* = \arg \max_a Q(s_{t+1}, a) \quad (14.26)$$

The ideal rule for updating Q :

$$Q(s_t, a_t) = Q(s_t, a_t) + \alpha(r + \gamma Q(s_{t+1}, a_{t+1})) - Q(s_t, a_t) \quad (14.27)$$

This can be given in terms of reward as:

$$Q(s_t, a_t) = Q(s_t, a_t) + \alpha(r_{t+1} + \gamma Q(s_{t+1}, a_{t+1})) - Q(s_t, a_t) \quad (14.28)$$

Where, α and γ are learning rate and discount factor, respectively.

The SARSA algorithm is given below.

Algorithm 14.8: SARSA

1. For all states and actions, set $Q(s, a) = 0$.
2. Initialize α , ϵ and γ .
 Initialize the state s_t .
 Choose action a_t based on selection policy ϵ -greedy strategy and execute it.
 Get a new state s_{t+1} , observe the reward r_{t+1} .
 Choose action a_{t+1} from Q based on ϵ -Greedy procedure and execute it.
 Update Action-value function of Eq. (14.28).
3. Update the values $s_t = s_{t+1}$, $a_t = a_{t+1}$.
4. If s' is terminal node, then start a new episode else repeat above steps.

The parameters of the SARSA algorithm as same as the Q -learning algorithm, that is, learning rate and discount factor.